
ПОДГОТОВКА К ЭКЗАМЕНУ

Конспекты лекций

Распределённые и облачные базы данных

Курс РБД · 2025/2026 учебный год
Группа М09КИИ-25
Материалы с занятий

Оформлено для подготовки к экзамену · со схемами по ключевым темам

Содержание

- Базы данных: файл, СУБД, ключи
- Физический уровень хранения и WAL
- Типы моделей данных
- Архитектура ANSI/SPARC: три уровня и независимость данных
- Слои типовой СУБД: путь запроса
- Нормализация
- Буферизация
- Хеширование в СУБД
- Кластеризация таблицы
- Соединения, сессии и транзакции
- Свойства транзакций: ACID
- Одноранговые системы, прозрачность, CAP
- Агрегаты для центра и репликация
- Фрагментация, размещение и разбор распределённого запроса
- Распределённые транзакции, контроль параллелизма, облачные базы
- Облако на своей инфраструктуре и проектирование объектов КИИ
- Безопасность баз данных: субъекты, политики, DAC, представления, RLS
- Модели управления доступом: Белла — Лападулы, Биба, RBAC
- Распределённые и облачные базы данных — сводный конспект

Часть I. Основания

Часть II. Распределённые и облачные базы

Часть III. Безопасность

Базы данных: файл, СУБД, ключи

Контекст курса

- Технологии баз данных рассматриваются в разрезе КИИ (критической информационной инфраструктуры) и безопасности.
- Astra Linux — российская сертифицированная ОС на базе Debian; сертифицирована ФСТЭК, RedOS — дополнительно ФСБ.
- Сверх базовых прав в Astra есть мандатный доступ: без соответствующей мандатной метки даже root не получит доступ.
- Использование штатных репозиторий и библиотек даёт контролируемую цепочку поставки: обновления под контролем ведомств, отсутствие закладок.
- Проблема перехода с импортных решений — не в самой ОС, а в прикладном софте, десятилетиями писавшемся под управление импортным железом.

Файл и база данных

- Файл — последовательность байтов с именем и правами, выданными операционной системой.
- База данных — совокупность связанных данных, снабжённая описанием и правилами доступа для подключающихся клиентов.
- СУБД — комплекс программных средств, обеспечивающий определение, хранение, поиск, обновление и защиту данных, поддерживающий их согласованность при параллельной работе пользователей и обеспечивающий восстановление после сбоя.
- Разница файла и базы не в объёме и не в структурированности: архив на 17 ГБ — тоже структурированный файл.
- Принципиальных отличий два: одновременный (конкурентный) доступ и соблюдение прав.
- При конкурентном доступе к простому файлу потеря изменений гарантирована.
- База может состоять из множества файлов или из одного: однофайловый пример — Microsoft Access, ценный скорее как оболочка и построитель отчётов, чем как хранилище.
- Разруливать права удобнее в одном месте, а не разрешать каждому клиенту лезть к файлу напрямую.
- Восстановление после сбоя лектор назвал самым тяжёлым по ресурсам местом в проектировании СУБД.

Основные функции СУБД

- Взаимодействие с физическим слоем хранения.
- Поддержка логической структуры: внутренние правила связи данных.
- Контроль целостности и типизации сохраняемых и изменяемых данных.
- Управление циклом изменения, пересборки и сохранения данных: изменение не ложится на диск мгновенно.
- Реакция на события над данными — триггеры (есть не во всех СУБД).
- Контроль корректности первичных и внешних ключей.
- Управление сессиями и изоляцией при конкурентном доступе.

Триггеры сильно потребляют ресурсы и способны неявно менять поведение функций: в практике лектора встречались проекты, где от триггеров отказывались волевым решением. Отдельный случай — рекурсия, когда триггер вызывает сам себя.

Ключи

- Первичный ключ — поле-идентификатор, значение которого уникально во всей совокупности данных и позволяет отличить один набор данных от другого.
- Естественный составной ключ — уникальность, собранная из нескольких содержательных полей набора.
- Внешний ключ — поле, задающее связь набора с внешней совокупностью данных: значение обязано присутствовать в другом наборе.
- Первичный ключ — ID, текстовый UUID или иное, лишь бы уникальное.
- Составной естественный ключ ненадёжен: в банковском проекте совпали ФИО, дата рождения и город двух разных людей — второму зарубили открытие счёта из-за данных первого. Ключ расширили СНИЛС, уникальность которого гарантирует выдающий орган.
- Задачи внешнего ключа: связь в рамках модели, ускорение поиска, исключение ошибок ручного ввода («Новосибирск» в трёх написаниях).
- Внешний ключ обеспечивает целостность: удалить значение, на которое есть ссылки, база не даст — либо каскад, либо разрыв связи.
- Вместо NULL во внешнем ключе лучше отдельная запись-заглушка «не определено»: иначе логику придётся дублировать в каждом поиске.

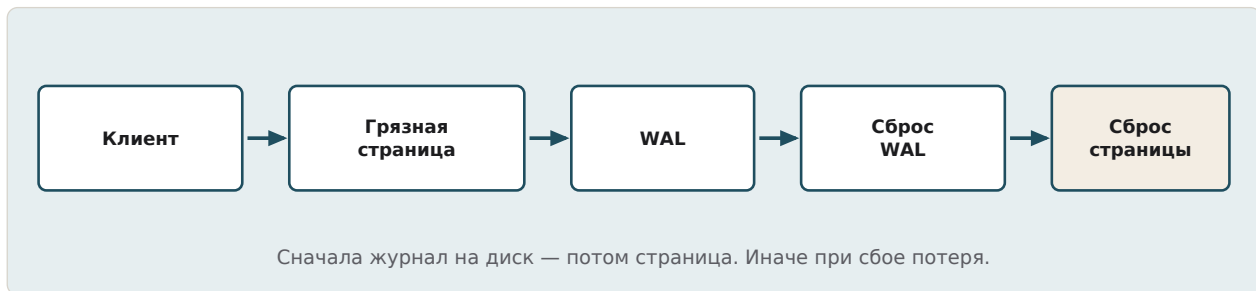
Конкурентный доступ и сессии

- Сессия определяет, как база реагирует на попытки конкурентного доступа — с точки зрения настроек безопасности и изоляции.
- Типовые коллизии: два клиента одновременно вставляют запись — кому достанется идентификатор 777, а кому 778; два клиента правят одно поле.
- Отдельная задача — как показывать данные, часть которых в режиме редактирования.
- Ресурсы делятся: тяжёлая выборка из полумиллиарда записей с сортировкой и подзапросами блокирует остальных.
- В распределённых системах это сложнее на порядок: узлы во Владивостоке и Москве имеют разное время доступа, а целостность нужно поддерживать совокупно.

Базовые термины

- Модель данных — язык структур, операций и ограничений.
- Схема — описание конкретной предметной области.
- Экземпляр — допустимое состояние данных в момент времени.
- Одни и те же по сути данные (человек) представляются по-разному в туризме, спорте, строительстве, банковском деле — форма определяется предметной областью, а не фантазией разработчика.

Физический уровень хранения и WAL



WAL: журнал опережает запись страницы

Организация хранения

- Задачи слоя хранения: организация страниц и файлов, размещение строк, освобождение места, сжатие, представления.
- Куча (heap) — простая последовательность записей: положить можно всё, но поиск конкретной записи — полный перебор.
- Индексированная структура — упорядоченное хранение указателей на физическое смещение записи в файле.
- Поиск по индексу: значения ключевых полей просчитаны заранее, по хешу сразу выдаётся расположение целевых данных.
- Цена индекса — дополнительная структура, занимающая место, и пересчёт при каждой вставке и обновлении.

Буфер и грязные страницы

- Клиент правит данные — изменения вносятся в страницу в буфере памяти, не на диск.
- Изменённая, но не сброшенная страница считается грязной.
- При запросе данных система сначала смотрит буфер; если данных нет — поднимает их с диска.
- Когда в буфере кончается место, вытесняется страница, к которой дольше всего не обращались (или по round-robin в упрощённом варианте).
- Только в момент вытеснения данные закрепляются на диске.

Сбрасывать каждое изменение сразу невыгодно: свежие данные, с которыми идёт текущая работа, обычно одни и те же, и десять пользователей будут править те же страницы. Напрягать дисковый носитель ради каждой промежуточной редакции незачем.

WAL — журнал, опережающий запись

- WAL (write-ahead log) — журнал, опережающий запись: изменение страницы нельзя считать устойчивым, пока запись журнала не сброшена на носитель.

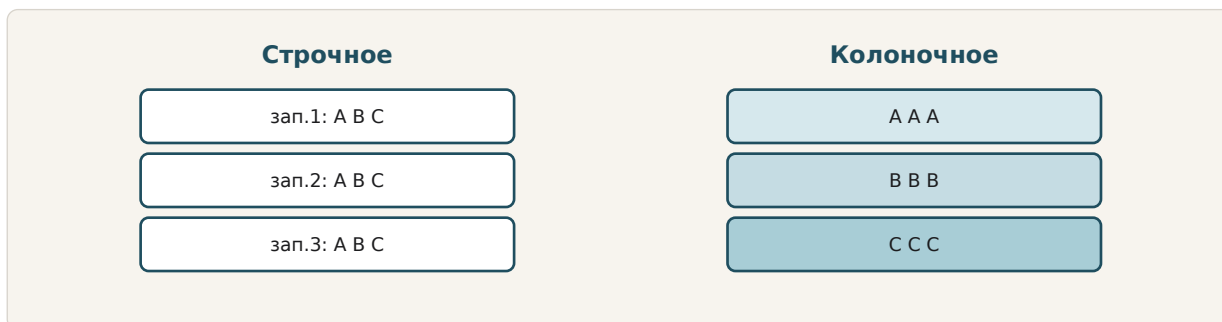
клиент → грязная страница в буфере → запись в WAL → сброс WAL на диск → (позже) сброс страницы на диск

- Закрепляется не страница, а запись журнала: страница не считается устойчивой, пока не закреплён журнал.
- При восстановлении система берёт точку закрепления (последняя физическая запись или иной маркер), пробегает весь WAL и накатывает изменения на соответствующие страницы.

- WAL тоже сбрасывается не по одной операции: 20 000 записей в секунду на носитель — это и расход ресурса диска, и время, особенно на HDD.
- Интервал между сбросами WAL — это допустимые потери; по мнению лектора, они должны стремиться к нулю.
- Позиция лектора: подход «потеряем три минуты данных, ничего страшного» неприемлем — для банковских транзакций, транспорта, реактора три минуты неизвестности недопустимы.

Лекция 1.2 · 2 сентября · 10:20–11:40 · Лекция · Альфа 5.12.2 (комп)

Типы моделей данных



Строчное и колоночное хранение

- Корректный термин — типы моделей данных, а не «типы баз данных»: база — просто совокупность упорядоченных связанных данных, а модель определяет их внутреннюю организацию.
- Модель задаёт единицу хранения, естественные операции и ограничения.
- Перечень моделей: реляционная, объектная, объектно-реляционная, документная, ключ-значение, колоночная, графовая, временные ряды, поисковая.
- Универсально лучшей базы не бывает — бывает наиболее подходящая под задачу.
- Часто эффективнее разделить хранение на несколько баз, используя сильные стороны каждой, и связать их уже на уровне информационной системы.

Реляционная

- Описывает данные как набор отношений: множества кортежей одной структуры с атрибутами, связанных с кортежами других отношений.
- Сила — декларативная целостность: то, что прописано в структуре, действует на все единицы хранения.
- Каждая запись имеет строго определённый набор полей строго определённого типа.
- Внешний ключ прямо кодирует обязательность отношения.
- Поля с неопределённым типом («либо строка, либо число») лектор считает извращением: дороже по времени и накладным ресурсам, проверять корректность приходится триггером. Лучше завести два поля.

Объектная и объектно-реляционная

- Объектная модель хранит идентичность, наследование и методы.
- Чистых объектных баз практически не применяют: совокупность объектов без отслеживания связей никому не нужна.
- Объектно-реляционная — гибрид: хранятся объекты и отношения между ними.
- Большинство современных «реляционных» баз де-факто объектно-реляционные: тип JSON хранится не строкой, а отдельным типом (PostgreSQL, Oracle).

Документная

- Хранит самоописываемые JSON-агрегаты с гибкой схемой.
- Отличие от объектной — нет наследования: каждая запись в пределах одной коллекции может быть уникальна по составу.
- Плюс — незаполненные поля не занимают места: в реляционной модели незаполненный `int` всё равно резервирует 4–8 байт на каждую строку.
- Минус — самодекларация: имя поля повторяется в каждой записи. Поле `uuid` — это минимум 7 лишних символов (имя, кавычки, разделитель) на запись.

- Практический вывод: максимально заполненные поля держать в реляционной части, разреженные — в документной. Это и есть один из случаев естественного распределения базы.

Ключ-значение

- Даёт скорость точечного доступа по известному ключу; сам ключ, как правило, не несёт смысловой нагрузки и не относится к хранимым данным.
- В архитектуре КИИ применяется параллельно с основной базой как кэш и координатор коротких блокировок.
- Возможен вариант, когда WAL и грязные страницы выносятся в отдельную быструю базу ключ-значение под небольшое число объектов.
- Требует чёткой программной связки: баланс «больше храним или больше обрабатываем» — постоянная развилка разработчика.

Колоночная

строчное: [зап.1: A B C][зап.2: A B C][зап.3: A B C]
 колоночное: [A A A][B B B][C C C]

- Подряд хранятся значения одной колонки — таблица фактически переворачивается.
- В строчном хранении смещение приходится вычислять по типизации каждого поля; в колоночном шаг фиксирован и известен заранее.
- Лучше сжимается: рядом лежат однотипные значения одной величины.
- Быстрее точечный апдейт одного значения.
- Хранится блоками на разных страницах: блок char и блок int64 имеют разный размер шага.

Критерий	Колоночное подходит	Колоночное проигрывает
Характер данных	холодные, редко изменяемые	активно изменяемые
Нагрузка	чтение и выборка по полю	частая запись
Ширина структуры	немного полей	200 полей — много смещений

Оговорка лектора: за 25 лет практики прикладного применения колоночного хранения он не встречал — холодные данные проще выгружать в файлы и стримить. Возможная ниша — научные базы.

Графовая

- Описывает вершины и рёбра; идеальна для хранения взаимосвязей.
- Лучше всего проявляет себя на связях многие ко многим: «один к одному» и «один ко многим» нормально ложатся в реляционную модель.
- Реализация «многие ко многим» реляционно порождает разрастающиеся таблицы связей: при правке нужно найти старые связи, сверить с новым набором, часть удалить, часть создать.
- Живой пример: в строительном проекте объекты, работники, менеджеры и типы работ вынесли в графовую базу — код сократился примерно в полтора раза. До этого таблица связей переполнилась за полгода на этапе внедрения.
- Типовое разделение в КИИ: идентификатор и описание устройства — в реляционной модели, вся топология связей — в графовой.

Временные ряды

- Заточена под поток «время — метрика — значение», идеальна для съёма телеметрии.

- Сложность не в записи, а в выборке: собрать данные тысяч датчиков во временном окне отсортированными — заметная нагрузка на обычную базу.
- При частоте съёма раз в полсекунды телеметрию выносят на отдельную систему, желательно физическую, а не виртуальную.
- Критичный параметр — скорость записи: терять телеметрию нельзя, а виртуализация снижает отдачу носителя.
- Порядок величин из практики: 3 узла хранения и 3 оркестратора, около 9 физических серверов с NVMe на одну базу при 10–15-кратной штатной нагрузке и пиках до 50.

Поисковая

- Два варианта поиска: по индексу либо полным перебором.
- Индекс: выбранные поля (например, ФИО и дата рождения) просчитываются заранее, при запросе вычисляется хеш и сразу выдаётся расположение данных либо ответ «нет».
- Без индекса — перебрать всё, сравнить и выдать результат или «нет».
- Работа администратора БД в большой базе: жить в журнале, ловить операции с задержкой выше критической, отличать ошибку от новой фичи и заводить под повторяющиеся запросы дополнительные индексы.
- Каждый дополнительный индекс — дополнительные расходы на хранение.

Классы систем по назначению

- OLTP (Online Transaction Processing) — нагрузка короткими транзакциями чтения и записи текущего состояния.
- OLAP (Online Analytical Processing) — аналитическая нагрузка: обработка больших объёмов, агрегирование и отчёты.
- HTAP (Hybrid Transactional and Analytical Processing) — попытка совместить транзакции и аналитику в одной системе.

Критерий	OLTP	OLAP
Нагрузка	короткие транзакции	большие объёмы, агрегирование
Операции	чтение и запись	преимущественно чтение
Требования	низкая задержка, строгая целостность, предсказуемая изоляция	отчёты за длительный период
Размещение	первичный узел	реплика или отдельный сервер

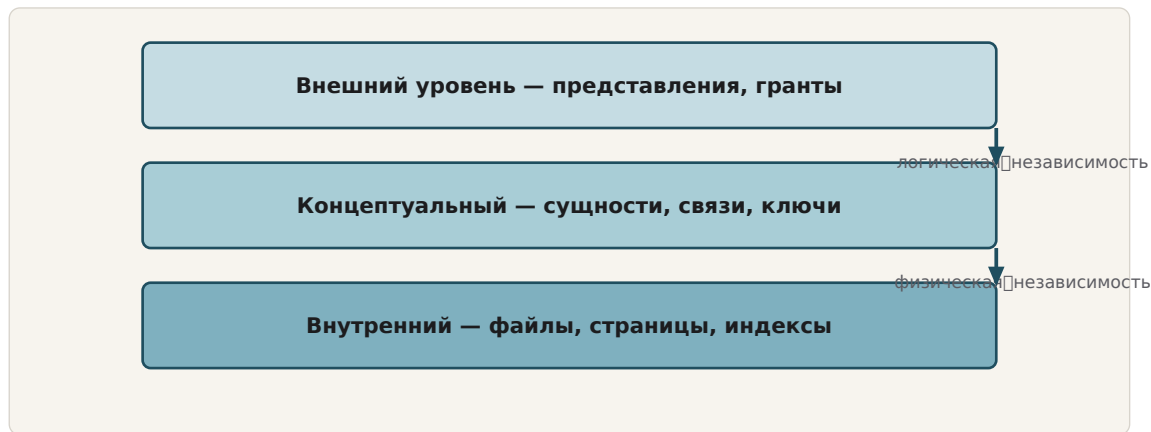
- Для OLTP потеря данных в идеале нулевая; на объектах КИИ именно этот подход определяет оперативный контур.
- Аналитику выносят на отдельный сервер, а не просто в отдельную базу: процессор и память у одной машины общие, загрузили в одном месте — нестача в другом.
- HTAP требует роста ресурсов; практичнее взять два сервера, каждый под своё дело.

[оперативная система] —пишет→ [OLTP-база]

- | ночная выгрузка за день
- ▼
- [аналитический сервер] ←отчёты— [директор]
- Пример из практики: в информационной системе строительной организации сбор отчёта за 3–4 месяца вешал всю систему. Аналитику вынесли на отдельный виртуальный сервер, куда каждый вечер сливались данные за день; отчёт собирался из готовой аналитики.

- Разделение возникает естественным путём — как только это почувствовали, запрос на отдельный сервер приходит от руководства сам. Совет лектора: написать официальный запрос заранее, чтобы осталась бумажка.

Архитектура ANSI/SPARC: три уровня и независимость данных



ANSI/SPARC: три уровня независимости

- ANSI/SPARC — международный комитет стандартизации, предложивший в 1975 году эталонную архитектуру систем баз данных.
- Это идеальная модель, а не описание конкретной СУБД: аналог многоуровневой модели сетевого стека, которой все придерживаются приблизительно.
- Реальные системы дробят или объединяют уровни под свои задачи.
- Всё описанное относится к реляционным и объектно-реляционным базам; в MongoDB представлений как понятия нет, в графовых — своя логика.

[внешний уровень] представления, гранты — что видит потребитель

- ▲ логическая независимость

[концептуальный] сущности, связи, ограничения, ключи

- ▲ физическая независимость

[внутренний] файлы, страницы, индексы, сжатие

Внешний уровень

- Одни и те же данные выдаются каждому потребителю в виде, отвечающем его задаче: преподаватель не видит паспортных данных студента, учебный отдел видит.
- Технически это схемы данных и гранты на доступ к таблицам и полям.
- Представление может не только ограничивать, но и преобразовывать: переименовать поля, разбить или слить их, собрать данные из нескольких таблиц.
- С точки зрения КИИ и информационной безопасности это дополнительный уровень защиты данных.
- Для надёжности это контракт между контурами: внутреннюю структуру можно менять, и внешний потребитель об этом не узнает.
- Плата за удобство — дополнительное потребление ресурсов: данные не отдаются напрямую, сначала собираются и обрабатываются.

Концептуальный уровень

- Это картинка предметной области: отображение реальности на жёстко детерминированную систему.

- Содержит сущности, связи, ограничения, глобальные имена, ключи.
- Здесь принимаются все основные технологические решения: что и как резать, разделять, реплицировать.
- Клиенту на внешнем уровне должно быть безразлично, две ли это копии базы, пара шардов или схема резервирования 3-2-1.

Внутренний уровень

- Описывает файлы, страницы, порядок записей, физические индексы, сжатие, пространство имён.
- Всё, что обеспечивает физическую работу с данными.

Независимость данных

- Физическая независимость — изменения внутреннего уровня не затрагивают концептуальный и внешний уровни.
- Логическая независимость — изменения концептуальной схемы, не отнимающие информацию у потребителя, не затрагивают внешний уровень.
- Разделение на шарды или создание реплики не имеет права накладывать ограничения на верхние уровни.
- Пример логической независимости: адрес перестали хранить в одном поле и разложили на город, улицу, дом — представление осталось прежним.
- Независимость работает снизу вверх: изменения на уровне не должны отражаться на слоях выше.

Независимость в распределённых системах

- Добавляется независимость от расположения, фрагментации и репликации.
- Приложение не должно работать с конкретным физическим узлом.
- Привязка клиента к одному узлу останавливает работу при его отказе; через общий брокер работа лишь замедлится, уйдя на удалённую реплику.
- Второй довод против привязки — сохранность данных: разъехавшиеся реплики при слиянии дают грязные или потерянные записи.

Слои типовой СУБД: путь запроса



Слои СУБД: путь запроса

Разбор идёт на примере PostgreSQL, с которым предстоит работать. В других системах слои могут быть переименованы — это снова идеальная схема.

запрос → [парсер] → [оптимизатор] → [менеджер транзакций] → [движок выполнения] → [передача результата] → клиент

- Парсер — строит дерево разбора запроса и проводит семантический анализ, сверяя имена с каталогом и правами.
- Подслой переписывания — переходит от представлений к физическим данным, сверяясь с политикой грантов; здесь внешний уровень пересекается с концептуальным.
- Оптимизатор — анализирует логический план и по статистике каталога находит самый эффективный вариант выполнения.
- Менеджер транзакций — выполняет блокировки, реализует атомарность и изоляцию.
- Движок выполнения — берёт оптимизированные команды, работает с физическими и сетевыми структурами, собирает промежуточные результаты в итоговый набор.
- Передача результата — собранный набор уходит клиенту через сетевую подсистему.

Что учитывает оптимизатор

- Один и тот же запрос имеет разные способы выполнения: есть индекс или нет, последовательное сканирование или обход индекса.
- Стоимость вложенных циклов, хэшей, сортировок и обработок.
- Один узел — учитывается страничная память и циклы; несколько узлов — ещё и время отклика каждого.
- Статистика каталога индивидуальна для системы и со временем меняется.

Транзакции в распределённой системе

- В одиночной системе всё просто: заблокировали, записали, разблокировали.
- В распределённой операции должны пройти по всем узлам и в одном порядке.
- Оговорка: если узел не ответил (сеть легла, перегрузка), транзакция может считаться проведённой при фиксации на двух из трёх машинах, а третий узел подтянется после восстановления.

Почему подключение всегда сетевое

- Даже когда сервер приложения и сервер БД на одной машине, подключение делают через сеть — на localhost или внутренний IP.
- Причина: базу в любой момент могут вынести на отдельную физическую машину, и тогда меняется только адрес доступа, а не код.

Нормализация



Нормальные формы (схема)

- Нормализация — способ организации структуры хранимых данных, предназначенный для исключения дубликатов и повышения надёжности.
- Самому носителю безразлично, что и как на него сложить — проблемы начинаются при использовании.
- Нормализация — выжимка практики тех, кто прошёл по граблям человеческого фактора.
- На паре спрашивалось не перечисление форм, а что такое нормализация по сути.

Первая нормальная форма

- 1НФ — каждая ячейка содержит одно атомарное неделимое значение, нет повторяющихся групп, присутствует первичный ключ.
- Первичный ключ даёт уникальный идентификатор набора: даже при совпадении всех прочих полей набор отличим.
- Фамилия, имя и отчество в одном поле нарушают 1НФ; ответ вида и так, и так уже некорректен.
- Удобство — понятие двоякое: человеку удобно одно поле, обработке — три; вопрос всегда удобно кому.

Вторая нормальная форма

- 2НФ — все неключевые атрибуты полностью функционально зависят от первичного ключа.
- Адрес по 1НФ уже разбит минимум на три поля: город, улица, дом.
- Количество городов фиксировано, поэтому город выносится в отдельную таблицу, а в записи хранится идентификатор.
- Выбор из готового списка надёжнее ручного ввода: иначе появляются два разных написания одного города и опечатки.
- Пример из практики: в кредитных отделах держали вторичную таблицу неправильных написаний городов и улиц, чтобы сопоставлять введённое с верным значением.

Третья нормальная форма

- 3НФ — при соблюдении 2НФ исключены транзитивные (каскадные) зависимости.
- Пример: направление зависит не от студента напрямую, а от его группы.
- Направление определяется строго через группу и не вписывается в запись студента — иначе при смене возникнет ошибка в данных.
- Всего форм пять, но практически используются первые три.

Буферизация

- Есть пул страниц быстрого хранения, куда по востребованности поднимаются данные с физического носителя.
- Изменения вносятся в буфер, страницы помечаются как грязные.
- Дальше либо немедленный сброс на диск, либо запись отражается в WAL (журнале).
- При переполнении буфера страницы вытесняются по установленному принципу.

Критерий вытеснения

Как работает

Слабое место

По давности

Убирается страница, к которой дольше всех не обращались

Давняя страница может быть активно нужной

По востребованности

Дополнительный счётчик обращений к каждой странице

Требует ведения счётчиков

По кругу

Стирается очередная по порядку

Не учитывает полезность страницы

- Настройка механизма вытеснения — работа администратора баз данных: анализ работы движка, задержек, динамическое обновление параметров.
- Лектор отдельно отметил, что это профессия для психостойчивых людей.
- Отдельный администратор нужен, когда объём данных велик и серверов не один-два: при масштабировании с 20 источников до 2000 они начинают конфликтовать между собой.

Хеширование в СУБД

- Хеш — значение фиксированной длины, вычисляемое по набору данных любого размера, одностороннее и практически однозначное.
- Не путать с кэшем: кэш — сохранённое ради скорости доступа, хеш — вычисленное значение.
- Так как длина хеша ограничена, теоретически два разных набора могут дать один хеш.
- Ключевое требование: по известному хешу алгоритмически невозможно восстановить исходный набор.

Три роли хеша

- Hash index — метод доступа.
- Hash join и hash aggregate — алгоритмы исполнителя.
- Hash partition — нарезка отношений по функции от ключа.

Hash index

- Отвечает только на равенство, определяя адрес записи.
- Начиная с некоторой версии PostgreSQL журналируется и не перестраивается после сбоя — раньше перестройка была тяжёлой нагрузкой.
- Хеш не имеет понятия больше-меньше, поэтому для сортировки и диапазонного поиска эффективнее B-tree.
- Хеш выигрывает при уникальном поиске по длинным значениям: индекс по трём-четырёх полям бинарно сравнивать тяжело.
- Хеш-таблица размещает значения так, чтобы хеш находился быстро, а по нему — смещение физической записи от начала.

Hash join

- JOIN — объединение данных из разных таблиц по совпадению значений.
- Сканирует меньшую сторону в таблицу в памяти.
- Пробивает её большей стороной.
- Часть, не уместившуюся в рабочую память, сбрасывает во временный файл.
- Соединение двух отношений по 50 миллионов строк даёт несколько гигабайт сброса.
- Риск: забитая оперативная память и фактический отказ базы в обслуживании.
- Требование КИИ: подобные джойны на внешнем уровне недопустимы — аналитика выносится в отдельный внутренний контур на отдельный сервер.

Hash aggregate

- Строит хеш-таблицу для группировки; на малом количестве групп дешевле других способов соединения.
- Когда групп много, объём растёт, и оценка стоимости становится критичной.

Контроль по журналу

- База растёт постепенно, и сбросы во временные файлы видны в журнале.
- Появление сбросов на хеш-операциях — сигнал оптимизировать систему.
- Если тяжёлую функцию вызывает клиент, её выносят на отдельный аналитический узел.

Принцип для критической инфраструктуры

- Критические точки — не только ядерные реакторы, но и электросистемы, биржи, дорожные и авиационные системы.
- Любое зависание в такой системе — сигнал к немедленному устранению.
- Подобные нагрузки просчитывают заранее; если просчитать не удаётся — ставят дублирующий контур.
- Формула лектора: всё просчитать нельзя никогда, значит дублировать нужно всегда.

Вопросы с пары

Вопрос. Почему не разрешить клиенту работать напрямую со своей репликой, это же быстрее, чем через брокер?

Ответ. Технически быстрее, но быстрее не значит лучше: при жёсткой привязке к одному узлу его отказ останавливает всю работу, а через общий брокер работа лишь замедлится на удалённой реплике; плюс расходящиеся реплики при слиянии дают грязные и утраченные данные.

Вопрос. Почему стараются не отправлять вложенный запрос?

Ответ. Он выполняется рекурсивно, а альтернатива — временные таблицы: их создают в процессе выполнения, наполняют, а потом выбирают из них нужное. Это сильно бьёт по производительности и памяти, но иногда без этого не обойтись.

Кластеризация таблицы

- CLUSTER — процедура, которая переписывает таблицу в порядке выбранного индекса и обновляет указатели; в базовом варианте выполняется с блокировкой таблицы.
- Чтение из базы идёт страницами: читается страница, из неё выбираются нужные строки.
- Если 200 нужных записей размазаны по 200 страницам, будет 200 операций чтения; после кластеризации те же данные лежат подряд и читаются за одну-три страницы.
- Индекс для кластеризации не обязан быть уникальным — важно, чтобы он соответствовал самой частой группировке запросов.
- Любая вставка или изменение ключевых полей нарушает порядок: корреляция падает, и процедуру приходится повторять.
- Задача администратора — следить за корреляцией и определить порог, после которого запускается перекластеризация.

Лектор подчеркнул: кластеризация — не свойство модели данных. Заранее заложить порядок хранения в модель нельзя: данные пишутся подряд на следующее свободное место, зарезервировать пустое место под будущие строки СУБД не позволяет.

- Востребованный порядок со временем меняется, поэтому кластеризующий индекс выбирают по статистике запросов и при необходимости меняют.

Соединения, сессии и транзакции

Соединение и сессия

- Соединение — связка двух хостов, где хост задаётся парой «IP-адрес и порт» отправителя и получателя.
- Сессия — контекст соединения: пользователь, база, параметры подключения, подготовленные операторы, временные таблицы и состояние транзакций.
- Кроме сетевого соединения возможен прямой драйвер к базе, но его почти не используют: он мешает последующей миграции и шифрованию канала.

Транзакция

- Транзакция — единица работы внутри сессии, всегда завершающаяся фиксацией или откатом.
- Транзакция проходит всегда, даже для одиночной команды и даже если её не объявляли явно: операция либо принята, либо отвергнута.
- С точки зрения приложения транзакция — последовательность операций, неделимая по смыслу приложения, хотя для базы каждая из них самостоятельна.
- Явную транзакцию удобно использовать как мягкий откат: при исключении в программе база возвращается в исходное состояние, даже если обработка ошибок в коде неполна.
- Синтаксис различается по серверам (BEGIN TRANSACTION, START TRANSACTION), суть одинакова.
- В документных базах явного механизма транзакций может не быть — связность документов остаётся заботой приложения.
- Изменения внутри транзакции не видны другим сессиям до успешной фиксации (в пределах выбранного уровня изоляции).

Требования критической информационной инфраструктуры

- Транзакции должны быть ограничены по времени и принудительно обрываться, иначе длинная транзакция блокирует записи надолго.
- Отсюда практика: чтение формы вне транзакции — прочитали, соединение закрылось, система не блокируется.
- Многие приложения авторизуются в базе от имени приложения, а не пользователя: меньше коллизий, но вся проверка прав уходит на бэкенд, а база принимает всё.
- Для КИИ требуется отдельный пользователь базы на каждого клиента: пользователь бэкенда заводится вместе с пользователем базы, соединение идёт от его имени.
- Тогда изменение статуса клиента должно начинаться с изменения статуса в базе, и статус нужно перепроверять на каждой транзакции — иначе при живой сессии запрет доступа не сработает.

Хранимые функции и триггеры

- Хранимая функция — процедура, вызываемая внутри СУБД и работающая с данными без передачи их по сети на бэкенд.
- Триггер — та же функция, вызываемая системой автоматически на определённое событие.

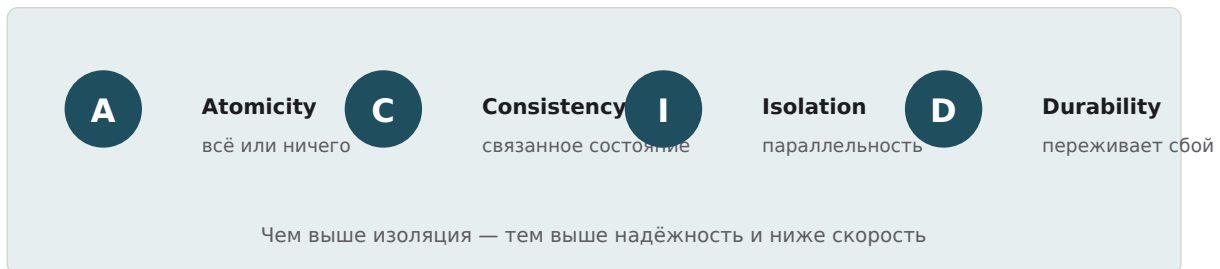
Привлекательность конструкции в том, что обработка идёт рядом с данными и не требует лишнего круга по сети, а мощный сервер базы обычно недогружен. Исторически это ещё и способ менять функционал без обновления клиента, когда клиент работает с базой напрямую, без бэкенда.

Почему их не любят в нагруженных системах

- Функции и триггеры не синхронизируются автоматически между узлами: их приходится обновлять руками на каждом сервере кластера.
- При динамическом поднятии узлов и обслуживании части баз обновление превращается в несогласованное состояние кластера.
- Триггер нельзя изменить — старый удаляют и добавляют новый.
- Для этого приходится переназначать делimiters SQL: внутри тела функции точка с запятой должна быть частью языка, а не разделителем команд. Если после обновления делimiters не вернули, все последующие запросы на сервере исполняются некорректно.
- Триггер, меняющий данные своей же таблицы, может вызывать сам себя — получается бесконечная рекурсия, которую нечем прервать.

Вывод лектора: в большинстве нагруженных систем база — это хранилище с упорядоченным доступом, а логика жёстко отделена от системы хранения.

Свойства транзакций: ACID



Свойства транзакций ACID

A – atomicity → атомарность
C – consistency → согласованность
I – isolation → изоляция
D – durability → долговечность

- Атомарность — все операции транзакции выполняются вместе или отвергаются вместе; после отката следов вмешательства в данные не остаётся.
- Единственное место, где видна отвергнутая попытка, — логи сервера (не WAL): именно там ищут причину отката.
- Срок хранения логов — вопрос не технический, а политики организации: на режимных объектах их хранят отдельно для расследования инцидентов, на обычных проектах лектор советует хотя бы пару недель.
- Согласованность — переход базы из одного связанного состояния в другое связанное. Пример нарушения: сохранённые персональные данные, не связанные ни с какой другой записью, хотя по внешнему ключу связь обязана быть.
- Изоляция — параллельные транзакции не должны нарушать иллюзию последовательного выполнения. На практике это управляемый набор разрешённых аномалий, а не полный запрет.
- Долговечность — после успешного коммита изменения переживают сбой и восстановление: данные легли либо в файлы, либо хотя бы в WAL. Для распределённой транзакции — на всех или на максимуме доступных серверов.

Уровни изоляции

- Идеал — сериализуемость: результат совпадает с каким-то последовательным выполнением тех же транзакций.
- Стандарт допускает и более слабые уровни, потому что при высокой конкуренции строгая сериализация стопорит систему.
- Аномалии, по которым различаются уровни: грязное чтение (dirty read), неповторяемое чтение (non-repeatable read), фантомное чтение (phantom read).
- Грязное чтение означает, что соседняя транзакция видит изменённые, но ещё не зафиксированные данные.
- Общая зависимость: чем выше уровень изоляции, тем выше надёжность и ниже скорость.

Точки сохранения

- Savepoint — именованная отметка внутри транзакции, к которой возможен частичный откат без отмены всей транзакции.
- В классическое определение транзакции не входит, это надстройка современных СУБД, и есть не у всех серверов.

- Смысл — не терять корректную часть работы из-за одной проблемной операции: например, дублирующийся телефон сохраняется как комментарий или в отдельную неподтверждённую таблицу, а остальные данные фиксируются.
- Так задаётся уровень допустимости аномальных сохранений: полный откат для большой распределённой системы слишком жесток.

Транзакции в распределённой системе

```
[клиент] → [брокер] → [узел 1] ✓
├→ [узел 2] ✓ → кворум набран → commit
└→ [узел 3] * → догоняет по журналам
```

- Транзакция считается выполненной, когда она выполнена на всех репликах либо на большинстве.
- Отставший узел уходит в режим промежуточного восстановления и по журналам догоняет транзакции, выполненные за время его недоступности.
- Конфликт «часть серверов приняла одну транзакцию, часть — другую» разрешается брокером, через который проходит операция; при нескольких брокерах их приходится согласовывать между собой.
- Пример проектирования: под базу закладывалось 9 серверов — три под хранение, три под брокер, три под резерв (?).

Транзакционный слой поверх нетранзакционной базы

- Есть быстрые документные серверы без поддержки транзакций.
- Разбавившийся на лекции опыт компании: такие серверы использовались как ноды, а поверх ставился лёгкий бэкенд — свой на каждой ноде, — выполнявший роль транзакционного механизма и контролировавший фиксацию.
- Задавался процент нод, которые должны подтвердить, чтобы транзакция считалась закрытой.
- Выигрыш — оттого, что универсальный транзакционный механизм СУБД тратит ресурсы на ветвления: проверку многочисленных настроек и уровней. Жёстко прописанный линейный алгоритм под конкретную задачу работает заметно быстрее.
- Отсюда общий вывод лектора: универсальной «лучшей базы» нет — каждая решает свои задачи со своими настройками.

Вопросы с пары

Вопрос. в чём главный минус авторизации в базе от имени приложения, а не клиента?

Ответ. вся проверка прав уходит на бэкенд, база принимает любые изменения; для критической инфраструктуры нужен свой пользователь базы на каждого клиента.

Вопрос. нельзя ли просто поднять отдельный инстанс под хранимую логику, раз ресурсы есть?

Ответ. так делают, но тогда рядом с каждым инстансом базы нужен свой обновляемый инстанс этого слоя, и при трёх и более частях базы схема перестаёт работать.

Вопрос. что будет, если две сессии одновременно вставляют один и тот же объект?

Ответ. вторую база отвергнет; для таких случаев транзакции над одним набором данных выполняются строго последовательно, что дорого при высокой параллельности.

Одноранговые системы, прозрачность, CAP



Теорема CAP

Одноранговые (P2P) базы

Фиксированных ролей нет: каждый узел хранит фрагменты, принимает запросы и пересылает соседям то, на что не может ответить сам. Точкой входа может быть любой узел; на клиенте держат список узлов по удалённости, недоступный сменяется следующим — работа продолжается и при отказе. Если данные распределены, принявший запрос узел рассылает части соседям и собирает общий ответ.

- В зеркальной схеме пересылка идёт по загруженности, в неоднородной — по консистентности.
- Минимальное число узлов берут из необходимости покрыть если не пиковую, то максимальную штатную нагрузку: держать десятикратный запас под пики экономически бессмысленно, в пик допускают рост времени отклика.

Проблема конфликтов

В P2P и в режиме мультимастера возможны расхождение версий и дублирование ключей, а решать конфликт некому. Разрешают по времени (кто раньше или кто позже), векторами версий либо иным заранее заданным правилом.

- Чистый одноранговый вариант без мастера опасен: два узла могут принять противоположные решения по одному объекту.

Практический компромисс — одноранговая схема с автоназначением мастера: в каждый момент кто-то отвечает за окончательное решение, при его отказе мастером становится другой узел. Чистый P2P оставляют для справочников и исторической информации.

Прозрачность (ось 4 классификации)

- Прозрачность — способность скрыть от приложения то, что база распределена. Для клиента разницы быть не должно.
- Location transparency — скрыт узел хранения.
- Fragmentation transparency — скрыто то, что разные части данных лежат на разных узлах.
- Replication transparency — скрыто наличие копий.
- Concurrency transparency — скрыты чужие транзакции, всё выглядит как обычный ACID, без ручных распределённых блокировок.
- Failure transparency — отказ узла отдаётся как обычная локальная ошибка, без указания сбойного узла. Если из трёх узлов транзакция прошла на двух, третий догонит при

восстановлении.

- Если распределение разруливается на бэкенде — это не ошибка реализации, а ошибка логики: база склеивается с бизнес-логикой, слои перестают быть независимыми, дальше легаси и костыли.

Полная прозрачность не бесплатна и достижима не всегда, но нарушать её «ради скорости» — путь к неуправляемой системе, особенно когда изменения не документируются.

Теорема CAP

- Распределённая система не может одновременно давать линейную согласованность, доступность и устойчивость к разделению сети.

Чем-то придётся жертвовать; алгоритмы, предложенные на основе теоремы, смотреть отдельно.

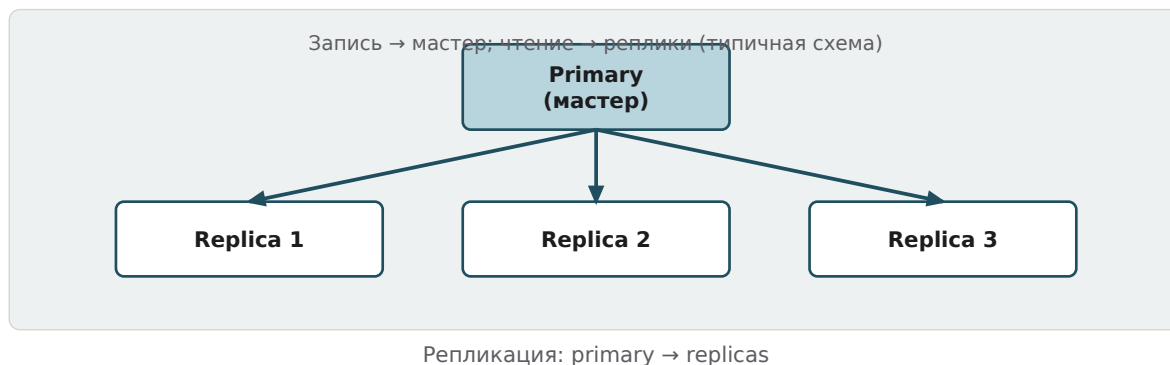
Почему для КИИ это не только про технику

- Временное решение, о котором забыли и не задокументировали, стоит объекту сертификации.
- Худший исход нарушений — не потеря данных и денег, а человеческие жертвы и экологические последствия. Отсюда цена не только в сложности и трудозатратах, но и в безопасности.

Про индивидуальный проект

Любую систему разбирают с четырёх сторон: из чего состоит, как части связаны, как взаимодействуют и что скрывают. В проекте надо обосновать, что и почему разделяется, как это связано на уровне базы и в чём именно распределённость.

Агрегаты для центра и репликация



Почему агрегаты держат избыточно

- Посчитали и отдали в центр — дальше за ними не ходят, лишнего трафика нет, персональные данные не уходят.
- Неизбыточный агрегат, лежащий только у себя, порождает постоянные обращения извне.
- Если схема оставляет центр без агрегатов, он начинает тянуть сырые данные, и нагрузка растёт экспоненциально.

Эти же агрегаты обычно становятся экспортными данными для федеративного объединения разнородных баз.

Репликация

- Репликация — поддержание копий одних и тех же данных на нескольких узлах, где изменение на одном рано или поздно отражается на других. Реплика — это копия фрагмента.

Отличие от фрагментации: при фрагментации на узлах лежат разные данные, при репликации — одни и те же. Реплицировать можно и отдельные фрагменты, если у них своя степень важности.

Три цели

- Доступность — при отказе главного одна из копий становится главной.
- Сокращение издержек — запрос выполняется на ближайшей реплике.
- Разделение нагрузки — тяжёлая аналитика уходит на выделенную реплику и не задерживает остальных.

Синхронная и асинхронная

- Асинхронная: транзакция считается зафиксированной на ведущем до подтверждения ведомого. Быстро, но точка восстановления ненулевая — при отказе часть зафиксированных транзакций может не оказаться на реплике. Для критичных контуров (журнал переключения оборудования, ККМ) недопустимо.
- Синхронная: фиксация ждёт подтверждения от одной или нескольких реплик. Commit не вернётся, пока журнал не достиг оговорённого рубежа.
- На реплику передаётся журнал, а не сам запрос: журнал — это уже обработанные и выверенные изменения страниц.

Физическая и логическая

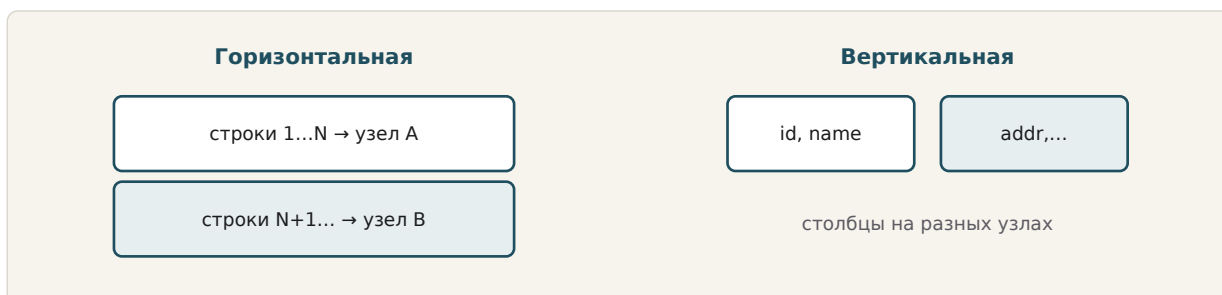
- Физическая копирует журнал предзаписи на уровне страниц и записей, ведомый получается побайтовой копией. Передача либо файлами журнала (потери в пределах сегмента), либо потоковая (канал занят постоянно, потери минимальны).
- Логическая декодирует изменения и применяет их как отдельные операции; ведомый может отличаться версией и в допустимых пределах схемой, можно реплицировать только часть данных.

Для аварийного переключения берут физическую, для интеграционных решений и витрин с урезанным набором данных — логическую.

Топологии

- Ведущий — горячий резерв. Пишем только в ведущий, ведомые получают журнал и обслуживают чтение. Конфликтов записи нет, переключение практически ручное. Базовый режим для оперативного контура.
- Мультимастер. Запись на нескольких узлах: производительность выше, но появляются конфликты — по времени, по приоритету узла или вручную администратором. Возможны расхождения в учёте.
- Каскадная. Промежуточный узел-ретранслятор получает журнал от ведущего и раздаёт нижестоящим: разгружает ведущего и экономит межплощадочный трафик.

Фрагментация, размещение и разбор распределённого запроса



Фрагментация данных

Около 90% трафика локализуется внутри подразделения, общим остаётся меньшая часть. Фрагмент удобен и как самостоятельная единица резервирования: пять фрагментов по терабайту копируются параллельно и быстрее, чем одна база на пять терабайт.

- Масштаб для ощущения: заливка 3 ТБ в университетское облако по сети оценивалась в ~147 дней; физическим диском прямо в ноду — четверо суток.

Горизонтальная фрагментация

Деление по кортежам (строкам). Объединение всех фрагментов должно покрывать исходное множество. Границу деления определяют эмпирически по структуре логов запросов — так, чтобы большая часть обращений попадала в один фрагмент. Обычно совпадает с административными и географическими границами.

Производная фрагментация

Дочерние отношения режутся по тому же плану, что и родительские.

- Локальный администратор не имеет права нарезать базу дополнительно без согласования с глобальным уровнем: централизованное обновление может не пережить самодеятельность.
- С точки зрения ИБ лишнее деление добавляет вектор компрометации при передаче.

Вертикальная фрагментация

Проекция отношения на подмножество атрибутов: данные о сущности лежат «слоями», каждый фрагмент содержит ключ для сборки.

- Это не нарушение третьей нормальной формы: соблюдается условие соединения без потерь, реконструкция идёт через natural join по ключу.

Отступление про natural join: он соединяет по совпадающим именам полей, а не по объявленным ключам. Поэтому требует строго индивидуализированных имён в каждой таблице, что ограничивает универсальность кода — на практике преподаватель использовал его считаное число раз.

Вертикальная фрагментация хорошо ложится на мандатный доступ: грантовая система выдаёт всё или ничего, мандатная — доступ к отдельным слоям.

Гибридная фрагментация

Композиция горизонтальной и вертикальной в любом порядке. Если первичной была вертикальная, каждый слой может дальше делиться горизонтально независимо от других — структуру данных это не ломает.

Два вопроса, которые в КИИ выводят на выбор способа: чей это объект и кому этот объект доступен.

Корректность фрагментации — три условия

- Полнота — условия разделения покрывают всё исходное множество.
- Восстановимость — есть реляционное выражение, собирающее исходное отношение: объединение для горизонтальной, соединение для вертикальной. Именно это отличает РБД от набора параллельных баз, склеиваемых прикладным слоем.
- Непересечение — для горизонтальной кортеж попадает ровно в один фрагмент; для вертикальной атрибуты не дублируются, кроме ключа сборки.

Размещение фрагментов

Фрагментация отвечает на вопрос «как режем», размещение — «где и в скольких экземплярах лежит».

- Неизбыточное: каждый узел хранит один фрагмент. Плюсы — меньший объём хранения и отсутствие проблем согласования. Минус — отказ узла подвешивает фрагмент, и работа встала.
- Избыточное: фрагменты на нескольких узлах. Плюс — доступность и надёжность; минус — нужен протокол согласования копий с кворумом на запись (большинство узлов либо все, если система параноидальная).

Кроме географии и надёжности на разбиение влияют организационные и правовые требования:

- не хранить персональные данные в аналитическом контуре;
- не выводить технологический поток без агрегации;
- иметь локальную копию, достаточную для работы при обрыве связи с центром.
- Модель данных с разграничением доступа согласуется с юридической службой до реализации. Есть подпись — работаем; нет — «технически всё верно, но закон нарушен».

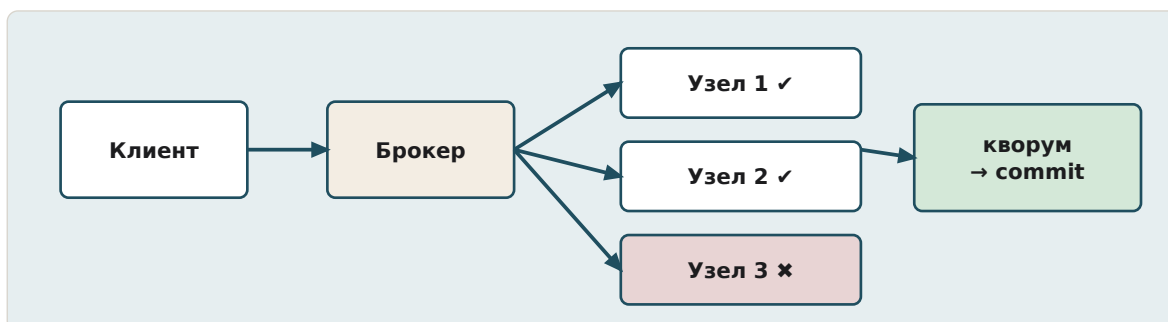
Обработка распределённого запроса

Клиент отправляет обычный SQL-запрос «чёрному ящику». Далее:

- Декомпозиция — запрос переводится в дерево реляционной алгебры над глобальными отношениями.
- Подстановка определений фрагментов — дерево становится деревом над частями данных, каждая ветвь разрешается отдельно.
- Локализация — из обработки выбрасываются фрагменты, чьи предикаты противоречат предикату запроса: фрагмент чужого региона не опрашивается.
- Проталкивание (pushdown) — селекции и обрезка столбцов переносятся ближе к узлу с фрагментом, чтобы по сети шли уже отфильтрованные данные.
- Выбор места сборки — оптимизатор решает, где выполнять итоговое соединение: на одном из задействованных узлов, на наименее загруженном или в центре.

Критерии выбора: процессорные единицы, страницы памяти, объём передачи по сети (локальная и глобальная сеть считаются по-разному), плюс требования надёжности и безопасности — перегон данных в чужой контур ради разгрузки должен быть запрещён.

Распределённые транзакции, контроль параллелизма, облачные базы



Распределённая транзакция и кворум

Каждый узел хранит свою часть данных и свой журнал. Координатор ведёт общую логику и принимает итоговое решение — commit или rollback.

Двухфазная фиксация (2PC)

- Фаза подготовки: координатор опрашивает участников; готовый записывает в устойчивое хранилище состояние `prepared` — ресурсы удержаны, самостоятельно ни откатить, ни зафиксировать нельзя.
- Фаза решения: все ответили положительно — рассылается `commit`; хоть один отказал или не ответил в срок — общий откат.
- Главная беда 2PC — блокировка при отказе координатора после входа участников в `prepared`: узел не знает состояния координатора и висит с занятыми ресурсами.

Лечат вмешательством администратора или временем жизни незавершённой транзакции — оба варианта чреваты расхождениями. Форсированные записи журнала и круговые сообщения на географически разнесённой системе слишком дороги.

Именно из-за этой проблемы и вырос механизм обмена журналами: главный контур планирует транзакции, ведомые применяют журнал, при сбое данные довосстанавливаются до консистентного состояния — не нужно тащить всю механику транзакций на каждый узел.

Трёхфазная фиксация (3PC)

Добавлено промежуточное явное состояние перед фиксацией, чтобы участник, не получивший решения, завершил транзакцию по тайм-ауту, а не блокировался навечно. Поведение при тайм-ауте настраивается: оборвать без фиксации либо (для параноидальных систем, где важно зафиксировать хотя бы где-то) выполнить локальный `commit`.

Плата — обязательный подробный план восстановления: после подъёма узел может обнаружить изменения у части соседей и должен знать, что с этим делать.

Контроль параллелизма в распределённой базе

Речь о слое поверх обычного контроля одиночной СУБД: управление параллелизмом, согласование реплик, поддержание целостности. Это не про информационную безопасность — она отдельно.

- Распределённый двухфазный захват блокировок. Транзакция делится на фазу накопления блокировок и фазу освобождения; блокировки удерживаются до конца ради

восстановимости. У каждого узла свой локальный менеджер блокировок, блокируются только ресурсы затронутых фрагментов. Глобальность означает: субтранзакция не отпускает блокировку на одном узле, пока работа идёт на другом. Захват автономен, освобождение управляется централизованно. Из-за долгих удержаний тяжёлую аналитику выносят на отдельные узлы.

- Метки времени. Конфликт разрешается по глобальному правилу — приоритет раннего или позднего. Схем много: правило Томаса для устаревшей записи, wait-die и другие.
- Сложность меток — синхронизация времени: брать штампы с независимых часов нельзя, за секунду проходят сотни транзакций, а запрос времени доходит по-разному. Отсюда логические часы (счётчики Лампорта) и гибридные методы.

Облачные базы данных

- Облачная база данных — база, развёрнутая и управляемая в облачной среде, а не на локальных серверах организации; доступ и операции идут через сеть, инфраструктуру поддерживает провайдер.

Идея выросла как сервис для небольших организаций: у большой базы стоимость аренды быстро перекрывает стоимость собственной инфраструктуры, особенно при росте потребляемых ресурсов. Плюсы — быстро развернуть, быстро масштабировать, не заниматься железом, питанием и сетью.

Модели

- Управляемая БД как сервис: провайдер ставит, обновляет, бэкапит и масштабирует; пользователю остаётся работа с данными и доступами.
- IaaS: берётся виртуальная машина, на неё ставится СУБД, администрирование целиком на заказчике — патчи, настройки, доступы, резервирование, масштабирование, как на физическом сервере.
- Гибрид: часть данных в своей инфраструктуре, часть в облаке. Причины: миграция, требования законодательства, публикация данных, федерация.
- Главный минус — инфраструктура не своя, и нужно абсолютное доверие провайдеру в вопросах надёжности и конфиденциальности. Никакой договор не вернёт утёкшие данные.

Облако на своей инфраструктуре и проектирование объектов КИИ

Облако не означает публичный интернет

Университетский пример: небольшой кластер, куда добавляются машины, ресурсы нарезаются виртуалками, к ним подключаются хранилища разных классов — NVMe тем, кому нужна скорость, жёсткие диски под холодное хранение.

- Изоляция и аудит: контуры разводятся на уровне платформы, ИБ контролирует систему целиком и не зависит от того, что отдельный человек наколбасит на своём наборе серверов.
- Масштабирование: докупили узел, подключили к кластеру, включили в распределение.
- Надёжность: нет жёсткой привязки к физическому устройству — при отказе нагрузка уходит на другие мощности, после ремонта узел возвращается прозрачно.
- Скорость операций: виртуальную машину быстрее клонировать и снести, чем поставить и разобрать физическую; неиспользуемый остаток физического сервера просто греет воздух, а в общем пуле достаётся соседям.
- Цена — сама прослойка виртуализации съедает порядка 20–25% ресурсов. Это закладывается в расчёт наравне с резервированием.

Два подхода к проектированию объекта

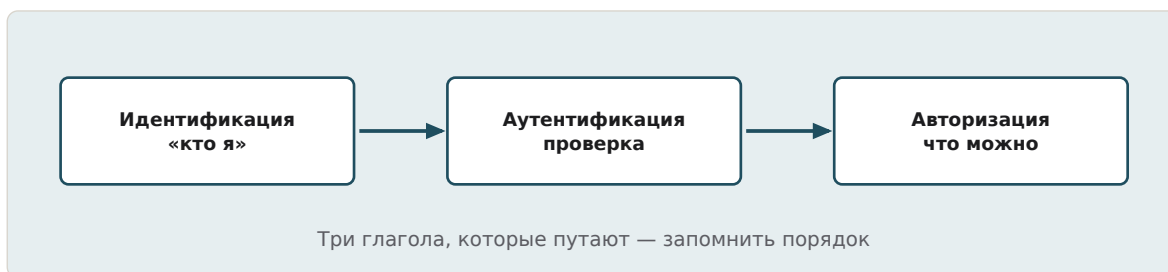
- Сверху вниз: что нужно → как это организовать → какие ресурсы и технологии.
- Снизу вверх, от места: работать будем в тундре — мало электричества, связь спутниковая, оборудование везти далеко, масштабироваться не выйдет. Это надо предусмотреть сразу.

На практике проходов несколько: схема балансируется между «что надо» и «где стоим», иногда туда-обратно не один раз.

- Бюджет — отдельная ловушка: распланировали на 130 миллионов, а потом выясняется, что есть 20.

Организационное с занятия: завтра работаем первой парой вместо второй, в четверг наоборот — второй вместо первой.

Безопасность баз данных: субъекты, политики, DAC, представления, RLS



Идентификация → аутентификация → авторизация

Нарушение правил доступа ломает два неочевидных свойства — конфиденциальность (люди видят лишнее) и целостность (зависимые части перестают разрешаться, структура рвётся в одном месте). Недоступность видна сразу, эти два — нет.

Базовые понятия

- Субъект доступа — сущность, от имени которой выполняется операция. Не обязательно человек: прикладная роль, служба репликации, задание резервного копирования.
- Объект доступа — то, на что направлена операция: кластер, база, схема, таблица, столбец, строка, последовательность, процедура, представление.
- Гранулярность — минимальная единица, на которую можно наложить правило.
- Право — разрешённый вид операции над объектом.
- Политика — правило, которое для пары «субъект — операция над объектом» выдаёт разрешение или запрет; может опираться на данные самой записи (пользователь правит только свои строки).
- Почему бэкап выполняется от имени задания, а не администратора: у администратора прав на сами данные может не быть вовсе — он ставит задания и наблюдает, но данные не читает.

Обратный, плохой случай — когда за одним субъектом стоит множество людей: внешнее приложение ходит в базу под общей учёткой, и автора действия потом не найти. Для интернет-магазина с игрушками неважно, для компонентов двойного назначения — уже нет.

Архитектура соединений — тоже часть модели доступа: доступ из внутренней сети и из внешней должен разграничиваться.

Где размещать политики

В правильно построенной распределённой базе — на верхнем слое, чтобы декомпозиция запроса сразу их учитывала и отсекала лишнее наверху. Выносить на бэкенд значит «очень быстро потерять». Оговорка: разграничение субъектов имеет смысл только там, где субъекты действительно разные — если всё идёт от одной учётки, политики бессмысленны.

Три свойства политики доступа

- Полнота — для каждой операции есть явное разрешение или явный запрет по умолчанию.
- Непротиворечивость — два правила не могут одновременно разрешать и запрещать без заданного порядка разрешения. Что главнее, личный доступ или групповой, однозначного ответа нет — это должно быть прописано.

- Проверяемость — аудитор может ответить, кто и почему получил разрешение. Отсюда требование: журнал должен переживать отказ всего остального.

В распределённой системе добавляется риск, что на разных узлах одной роли прописаны разные права.

Три глагола, которые путают

- Идентификация — заявление субъекта о том, кто он: имя, идентификатор, сертификат, ключ.
- Аутентификация — проверка этого заявления: пароль, подпись, сертификат, токен.
- Авторизация — решение, какие операции данному субъекту разрешены на данном объекте.

Аутентификация может быть общей, а авторизация в настоящей РБД — своя на каждом узле, и это нормально: на одной базе лежат пациенты, на другой персональные данные студентов, права там разные.

Дискреционное управление (DAC)

Решение принадлежит владельцу объекта, в основе матрица «субъект — объект — права», управляется GRANT и REVOKE. Создатель объекта становится владельцем, может выдавать права и отозвать их даже у себя.

- В Postgres право удалить или изменить объект не является выдаваемой привилегией — оно остаётся у владельца и передаётся только сменой владельца.
- Суперпользователь обходит все ограничения, поэтому раздавать администраторам суперправа — плохая практика. Администратор базы не должен управлять ролями пользователей.
- GRANT имеет два смысла: выдать привилегию на объект и выдать членство в роли. Группа — это роль без логина.
- Ловушка WITH GRANT OPTION: субъект передаёт доступ дальше, и при отзыве у него выданные им права могут остаться. Отзыв обязан учитывать зависимые выдачи.

Правильная схема для временной рабочей группы: создать роль, дать ей права, разрешить руководителю добавлять в неё сотрудников, а по завершении отозвать роль целиком — доступ пропадёт у всех разом, лишнего не останется.

Представления как ограничение видимости

Субъекту выдаётся грант на представление вместо базовой таблицы, прямое обращение к таблице запрещается, лишние столбцы в выборку не попадают.

- WITH CHECK OPTION ограничивает вставку и изменение: операция проходит, только если предикат истинен. Иначе субъект мог бы вставить запись, которую сам не увидит, но которая отобразится у другого.
- Представление с явно перечисленными полями иногда называют декорацией.
- Представление — не криптография. Суперпользователь его обходит, поэтому разграничение не отменяет журналирование.

Row-level security

Горизонтальное ограничение на уровне строк без отдельного представления под каждую роль: с добавлением роли пришлось бы плодить новый комплект представлений.

- Работает поверх обычного доступа к таблице; если политик нет — запрет по умолчанию.
- Исключения: владелец таблицы, суперпользователь и роли с BYPASSRLS.
- Политика задаётся выражением: USING для чтения, WITH CHECK для вставки и изменения (если не задан — совпадает с USING).

- Разрешающие политики складываются по «или», ограничивающие — по «и».

Остаточный риск: по ошибке внешнего ключа можно сделать вывод о существовании скрытых строк.

Модели управления доступом: Белла — Лападулы, Биба, RBAC



Мандатные модели доступа

Мандатная модель (Белла — Лападулы): конфиденциальность

Классы доступа жёстко разделены, решение принадлежит центральному администратору, а не владельцу таблицы.

- Не читать вверх: субъект не читает объекты класса выше своего.
- Не писать вниз: субъект не пишет в документы класса ниже своего, иначе секретное утечёт в несекретное.
- Двойственная ситуация: если поле документа имеет класс выше класса субъекта, он его не видит, но формально может туда писать. Система не даст затереть важные данные — она создаст низкоуровневую версию значения поля, а обладатель высокого доступа продолжит видеть полную картину.

Модель сложна в реализации. Postgres мандатное разделение не поддерживает: наложить его поверх грантовой схемы означает разводить контуры, и на длинной дистанции это становится серьёзной проблемой. Если задача возникла всерьёз — лучше один раз перейти на систему, изначально ориентированную на мандатный доступ.

Модель Биба: целостность

Работает зеркально: не читать вниз, не писать вверх. Высокоцелостный субъект не опирается на данные низкой целостности, низкоцелостный не изменяет высокоцелостные.

- Отчёт регулятору не берёт цифры из непроверенного документа.
- Нельзя глушить реактор по срабатыванию одного непроверенного датчика — сначала перепроверка.

Для КИИ целостность зачастую важнее конфиденциальности в плане обеспечения работы. На практике в одном центре обе модели уживаются: Биба — для контура управления, Белла — Лападулы — для контура расследования инцидентов.

Ролевая модель (RBAC)

- Роль — устойчивый набор полномочий, соответствующий должностной функции.
- Права назначаются роли, роль — пользователю. Смена должности сводится к смене роли, а не к переписыванию сотен грантов на объекты.
- Иерархия ролей позволяет наследовать права: старший дежурный наследует дежурного и добавляет контрольные функции.

- Разделение обязанностей: тот, кто создаёт задачу, и тот, кто её проверяет, — разные роли.
- У одного человека может быть несколько ролей, но противоречить друг другу они не должны: приоритета у ролей обычно нет, и конфликт разрешится непредсказуемо.
- В наследуемых иерархиях унаследованное разрешение можно заменить — например, «создавать наряд» на «контролировать наряд».

Распределённые и облачные базы данных — сводный конспект

Собрано из расшифровок всех занятий курса: 2 сентября (1.1, 1.2), 3 сентября (2.1), 7 сентября (4.1), 8 сентября (5.1, 5.2), 14 сентября (9.1, 9.2), 15 сентября (10.1, 10.2), 16 сентября (11.1, 11.2). Порядок тематический, номер занятия указан у каждого раздела. Первая часть — основания (что такое база, модели данных, устройство СУБД, транзакции), вторая — распределённость и облака, третья — безопасность.

СВОДНЫЙ КОНСПЕКТ

Часть I. Основания

1. База данных, файл и СУБД (1.1)

- Файл — последовательность байтов с именем и правами, выданными операционной системой. Больше в нём ничего нет.
- База данных — совокупность связанных данных, снабжённая описанием и правилами доступа.
- СУБД — комплекс программных средств, обеспечивающий определение, хранение, поиск, обновление и защиту данных, поддерживающий их согласованность при параллельной работе пользователей и обеспечивающий восстановление после сбоя.

База может лежать и в одном файле (пример — Microsoft Access, который после кризиса 2008 года неожиданно пошёл в дело как дешёвая оболочка к SQL Server), и в множестве. Принципиальная разница между файлом и базой не в объёме, а в двух вещах: одновременный (конкурентный) доступ и соблюдение прав. Вокруг конкурентного доступа и строится весь курс.

Что СУБД делает помимо хранения: следит за логической структурой и типизацией, контролирует целостность и корректность ключей, управляет циклом изменения и сброса данных на диск, реагирует на события через триггеры, изолирует параллельные сессии и восстанавливается после сбоя. Последнее преподаватель считает самым тяжёлым — не продумать, а по потреблению ресурсов.

Ключи. Первичный ключ уникально определяет запись в наборе. Естественный составной ключ (ФИО + город) кажется уникальным, но на практике им не является: в банковском проекте встретились два человека с полным совпадением ФИО, даты рождения и города, из-за чего второму зарубили открытие счёта по данным службы безопасности на первого. Лечится добавлением в ключ СНИЛС — гарантированно уникального идентификатора от внешнего органа. Внешний ключ задаёт связь с другим набором: он же обеспечивает целостность (запрет удаления того, на что ссылаются, либо каскадное поведение), ускоряет поиск и убирает человеческий фактор — «Новосибирск», «г. Новосибирск», «Новосибирск » выбором из справочника не появятся.

2. Журнал предзаписи (WAL) и буфер (1.1, 2.1)

- Write-Ahead Log — журнал, опережающий запись. Изменение страницы нельзя считать устойчивым, пока на диск не сброшена запись журнала.

Обратите внимание: сбрасывается не страница, а журнал. Как это работает: данные лежат в буфере страниц, изменения помечают страницу «грязной», на диск она сразу не идёт — свежие данные, как правило, редактируются многократно, и дёргать носитель на каждую правку бессмысленно. Когда буфер переполняется, вытесняется страница, к которой дольше всего не обращались (LRU), либо просто очередная по кругу; во многих системах к этому добавляется счётчик обращений, чтобы часто используемая страница не вылетела только из-за возраста. Вот в этот момент данные и закрепляются на диске.

Если между этими моментами случится сбой, грязные страницы потеряны. Восстановление идёт по WAL: от точки последнего закрепления журнал проигрывается заново и изменения накатываются на страницы. Сам WAL тоже сбрасывается не по одной операции — писать на носитель 20 тысяч раз в секунду значит убивать его ресурс, — и вот это окно и есть

допустимая потеря. Позиция преподавателя: окно должно стремиться к нулю. «Три минуты данных потеряно» — приемлемо для новостей и неприемлемо для банковских транзакций, диспетчеризации транспорта или реактора.

3. Модели данных (1.2)

Корректный термин — «типы моделей данных», а не «типы баз данных»: модель задаёт единицу хранения, естественные операции и ограничения. Лучшей модели не бывает, бывает подходящая под задачу — и главный навык курса именно в том, чтобы понимать, какие данные где хранить и как сочетать это в одной системе.

- Реляционная. Данные как набор отношений — множества кортежей одной структуры. Сила — декларативная целостность: у каждой записи строго определённый набор полей строго определённых типов. Нетипизированные поля, куда кладут то строку, то число, преподаватель называет извращением: дороже и по времени, и по хранению. Внешний ключ прямо кодирует обязательность связи; вместо NULL по внешнему ключу лучше держать отдельную запись-заглушку «не определено» — поиск будет чище.
- Объектная. Хранит идентичность, наследование и методы. В чистом виде практически не встречается: связи всё равно нужны.
- Объектно-реляционная. То, чем фактически давно являются Postgres и Oracle: реляционная база с полноценным типом JSON, то есть с хранением упакованных объектов. Чистых реляционных и чистых объектных баз почти не осталось.
- Документная. Самоописываемые JSON-агрегаты с гибкой схемой, наследования нет, таблиц нет — коллекции, и каждый документ может быть уникален по составу. Экономия на незаполненных полях против реляционной (где под каждый пустой INT всё равно резервируются байты) частично съедается тем, что каждый документ самодекларируем: имена полей повторяются в каждой записи. Отсюда естественный гибрид — плотно заполненное в реляционке, разреженное в документной. Это и есть одно из естественных распределений.
- Ключ-значение. Максимальная скорость по известному ключу, смысловой нагрузки ключ не несёт. В архитектуре КИИ обычно идёт рядом с основной базой как кэш и координатор коротких блокировок.
- Колоночная. Таблица «перевернута»: подряд лежит колонка. Смещение до нужного значения вычисляется без разбора типов каждого поля, данные одного типа лучше сжимаются, точечное обновление быстрее. Проигрыш — на выборках по многим полям и при широкой структуре (200 полей — уже провал). Годится для холодных, редко изменяемых данных; за 25 лет практики преподаватель ни разу не применил её в прикладной задаче — холодное проще упаковать в parquet.
- Графовая. Вершины и рёбра, идеальна для связей «многие ко многим». В реляционной такие связи выливаются в разрастающиеся таблицы связей, которые к тому же мучительно редактировать: найти старые, сверить с новыми, часть удалить, часть создать. В строительном проекте вынос всей топологии связей — объекты, люди, менеджеры, типы работ — в графовую базу сократил код примерно в полтора раза.
- Ряды (time series). Заточены под поток «время — метрика — значение», то есть под телеметрию. Сложность не в приёме данных, а в выборке и сборке во временном окне — это и нагружает обычную базу. Большую телеметрию выносят на отдельное, желательно физическое железо: важна скорость записи, а виртуализация её съедает, терять телеметрию нельзя.
- Поисковые. Про них — в разделе про индексы.

База знаний — не тип базы данных, а прикладное явление: внутри она может быть собрана из каких угодно моделей.

4. Три типа нагрузки: OLTP, OLAP, HTAP (1.2)

- OLTP — короткие транзакции чтения и записи текущего состояния. Нужны низкая задержка, строгая целостность, предсказуемая изоляция. На объектах КИИ это оперативный контур.
- OLAP — аналитическая нагрузка, большие объёмы, агрегация и отчёты, преимущественно чтение. Выносятся в реплику или отдельное хранилище, чтобы не конкурировать с оперативным контуром.
- HTAP — попытка совместить то и другое в одной системе. Оценка преподавателя: требует такого роста ресурсов, что проще взять два сервера, каждый под своё.

Живой пример: информационная система строительной организации, десятки тысяч операций в день. Когда директор запрашивал отчёт за 3–4 месяца, у всех вставала работа. Аналитику вынесли на отдельный виртуальный сервер, куда каждый вечер сливались данные за день и заранее считались агрегаты. Процесс естественный: пока OLTP тянет — живёте, как почувствовали — сразу отдельная машина под аналитику (именно машина, не просто отдельная база: диск, процессор и память у одной машины общие). Согласование такого обычно проходит легко, потому что просят сами руководители; служебную записку лучше написать заранее, как только стало понятно, что к этому идёт.

5. Трёхуровневая архитектура ANSI/SPARC (2.1)

Эталонная архитектура 1975 года — такая же условная модель, как семиуровневая модель сетевого стека: её придерживаются с оговорками, но она задаёт общий язык.

- Внешний уровень — представления потребителей. Одни и те же данные выдаются каждому в его виде: учебному отделу паспортные данные студента нужны, преподавателю нет. Технически это схемы, представления и гранты; представление может переименовывать, объединять и преобразовывать поля. Для КИИ это ещё и дополнительный уровень защиты, и контракт между контурами: внутреннюю структуру можно перекроить, а внешний потребитель об этом не узнает и ничего у него не сломается. Плата — дополнительное потребление ресурсов.
- Концептуальный уровень — картина предметной области: сущности, связи, ограничения, глобальные имена, ключи. Здесь принимаются все основные технологические решения: что и как резать, сколько копий, как разносить.
- Внутренний уровень — файлы, страницы, порядок записей, физические индексы, сжатие, пространства имён.

Независимость данных бывает двух видов: физическая (изменения внутреннего уровня не трогают концептуальный и внешний) и логическая (изменения концептуальной схемы не отнимают информацию у потребителя). Для распределённых систем сюда добавляются независимость от расположения, фрагментации и репликации: приложение не должно работать с конкретным физическим узлом.

Отдельно разобран соблазн «пусть каждый филиал ходит напрямую в свою реплику, это быстрее брокера». Быстрее не значит лучше: при жёсткой привязке к узлу его отказ останавливает работу филиала целиком, а через брокер работа лишь замедлится, уйдя на удалённую реплику.

6. Как устроена обработка запроса внутри СУБД (2.1)

- Парсер строит дерево разбора, проводит семантический анализ, сверяет имена с каталогом и правами. Здесь же подслой переписывания: обращения к представлениям разворачиваются в обращения к физическим данным с учётом политик.
- Оптимизатор анализирует логический план и по статистике каталога выбирает самый дешёвый вариант: последовательное сканирование или индекс, вложенные циклы, хэш или сортировка. Учитывает, один узел или несколько, — во втором случае ещё и время отклика каждого. Статистика индивидуальна и со временем меняется.

- Менеджер транзакций выполняет блокировки, обеспечивает атомарность и изоляцию. В распределённой системе с оговоркой: транзакция может считаться проведённой при фиксации на большинстве узлов, отставший догонит при восстановлении.
- Движок выполнения работает с физическими и сетевыми структурами, собирает промежуточные результаты в итоговый набор.
- Передача результата — почти всегда через сетевой стек, даже когда база и приложение на одной машине: так базу можно вынести на отдельный сервер, поменяв только адрес.

7. Нормализация (2.1)

Нормализация — способ организации структуры, нужный прежде всего для исключения дубликатов и повышения надёжности. Это «жестокая практика тех, кто натанцевался по граблям человеческого фактора».

- 1НФ: в каждой ячейке одно атомарное неделимое значение, нет повторяющихся групп, есть первичный ключ. Значит, ФИО в одном поле — уже нарушение: фамилия, имя и отчество должны лежать отдельно.
- 2НФ: все неключевые атрибуты полностью функционально зависят от первичного ключа. Город в адресе — не строка, а идентификатор из справочника. Иллюстрация: в банке держали отдельную таблицу неправильных написаний городов и улиц, чтобы сопоставлять то, что менеджеры вбили руками.
- 3НФ: при соблюдении 2НФ исключены транзитивные зависимости. Направление обучения определяется через группу, а не дублируется в записи студента.

8. Индексы, хэш и кластеризация (1.2, 2.1, 4.1)

Без индекса поиск — перебор всех записей. Индекс — заранее просчитанная структура, которая по значению ключа сразу отдаёт физическое расположение данных. Плата: место под индекс и его пересчёт при каждой вставке и обновлении.

- Хэш — значение фиксированной длины, вычисляемое по набору данных любого размера; по хэшу нельзя восстановить исходный набор, а два разных набора теоретически могут дать одинаковый хэш.

Три роли хэша в СУБД:

- Хэш-индекс отвечает на равенство. Понятий «больше/меньше» у хэша нет, поэтому для диапазонов и сортировки эффективнее B-tree, а хэш выигрывает на уникальном поиске и длинных составных ключах, которые тяжело сравнивать побайтово. Начиная с определённой версии PostgreSQL хэш-индекс журналируется и не перестраивается после сбоя — раньше пересчёт был заметной нагрузкой.
- Hash join сканирует меньшую сторону в таблицу в памяти и «пробивает» её большей. Если не помещается в рабочую память, лишнее сбрасывается во временный файл: соединение двух отношений по 50 млн строк — это несколько гигабайт сброса и риск положить базу. Отсюда требование КИИ: такие соединения на внешнем контуре недопустимы, аналитика — во внутренний контур.
- Hash aggregate и hash partition — агрегация и нарезка отношений по функции от ключа.

Кластеризация (4.1) — не свойство модели, а процедура: таблица переписывается в порядке индекса. Смысл в чтении группами: если 200 событий одного устройства размазаны по 200 страницам, это 200 операций чтения; после кластеризации те же данные лежат в двух-трёх страницах. Любая вставка порядок нарушает, корреляция падает, и администратор должен следить за порогом, после которого процедуру нужно повторить — возможно, уже по другому индексу, если изменилось то, что чаще запрашивают.

Отсюда же портрет работы администратора базы: жить в журналах, ловить задержки, понимать, ошибка это или новая фича, и оптимизировать под неё. Преподаватель отзывается об этом как об «адовом труде» для людей с исключительной

психоустойчивостью.

9. Сессии, транзакции, ACID и уровни изоляции (4.1)

Сессия — контекст соединения: связка двух хостов (IP и порт с каждой стороны) плюс субъект, база, параметры подключения, подготовленные операторы, временные таблицы и состояние транзакций.

- Транзакция — единица работы внутри сессии. С точки зрения приложения — последовательность операций над данными, которая является неделимой единицей.

Транзакция есть всегда, даже когда её не объявляют явно: одиночная операция тоже либо принимается, либо отвергается. Явную транзакцию преподаватель открывает при любых изменениях данных — это удобный механизм мягкого отката при исключении в программе.

ACID:

- Атомарность — операции применяются или отвергаются целиком; следов отменённой транзакции в данных не остаётся, причину отказа ищут в логах (не в WAL — в журнале операций).
- Согласованность — база переходит из одного связанного состояния в другое связанное. Не должно остаться, например, адреса, не связанного ни с одной записью.
- Изоляция — параллельные транзакции не ломают картину последовательного выполнения сверх явно разрешённого уровня. Идеал — сериализуемость; ACID допускает и более слабые уровни, потому что при высокой конкуренции строгий режим всё стопорит. Практическая формулировка: изоляция — это управляемый набор разрешённых аномалий (грязное чтение, неповторяемое чтение, фантомы), а не просто «запрещено».
- Долговечность — после успешного commit изменения переживут сбой: данные либо на диске, либо как минимум в журнале, откуда будут восстановлены. Для распределённой транзакции — на всех узлах либо на большинстве доступных.

Точки сохранения (savepoint) — именованная отметка внутри транзакции, к которой можно откатиться, не отменяя её целиком. Полезно, когда часть данных не прошла проверку (дублирующий телефон), а остальное терять нельзя: спорное значение уходит в комментарий или отдельную таблицу неподтверждённого.

Отдельное требование КИИ: транзакции ограничиваются по времени принудительно. Открытая на чтении формы транзакция способна заблокировать записи надолго — чтение формы должно быть вне транзакции.

Авторизация в базе: по приложению или по клиенту. Многие подключаются к базе одной учёткой приложения — меньше коллизий, проще транзакции. Минус в том, что вся защита переезжает на бэкенд, а база принимает всё подряд. Правильный для КИИ путь: заводить пользователя базы вместе с пользователем приложения и соединяться от его имени, тогда работает RLS. Отдельно про сессии: статус пользователя надо перепроверять на каждой операции, иначе отозванный доступ будет действовать до конца живой сессии.

Хранимые функции и триггеры. Соблазн понятен: обработка идёт рядом с данными, без лишнего круга по сети, и нагруженный сервер базы не простаивает. Но обновлять их приходится руками на каждом узле кластера, а при динамическом масштабировании или обслуживании части узлов это ломается. Триггер нельзя обновить — только удалить и создать заново, для чего приходится переназначать разделитель операторов; если процесс оборвался до возврата разделителя, некорректно отработают все последующие запросы на этом сервере. Плюс триггеры меняют данные и способны вызывать сами себя рекурсивно. Вывод: в нагруженных системах база — это хранилище и контроль доступа, логика живёт отдельно, что совпадает и с принципом разделения слоёв.

Интересный контрпример оттуда же: компания взяла быстрые документные серверы без поддержки транзакций и поставила рядом с каждой нодой лёгкий бэкенд, выполняющий роль транзакционного механизма с кворумом подтверждений. Выигрыш получился за счёт того, что универсальный транзакционный слой базы тратит много ресурсов на проверки настроек, а жёстко прописанный линейный алгоритм под конкретную задачу работает быстро. Мораль: универсально лучшей базы нет.

СВОДНЫЙ КОНСПЕКТ

Часть II. Распределённые и облачные базы

10. Что считается распределённой базой, а что нет (5.1)

- Распределённая база данных — логически единая база, данные и обработка которой размещены на нескольких узлах, связанных сетью, при наличии общей схемы и общего языка запросов.

Ключевое слово — «логически единая». Распределением занимается сама СУРБД, а не бизнес-логика приложения. Пользователь пишет запрос к глобальной схеме, система сама решает, какие фрагменты затронуть, как согласовать копии и как зафиксировать транзакцию.

Три кластера Postgres под одним брокером, к которому ходит бэкенд, — распределённая база. Три отдельных контура, которые разруливает бэкенд, — распределённая система, но не распределённая база.

Что часто путают с РБД, но ею не является:

- Несколько серверов с одинаковой схемой и прослойкой-маршрутизатором. Прослойка просто выбирает копию по региону или иному условию; целостность и уникальность ключей не обеспечиваются.
- Шардированное приложение без глобальной схемы. Ключ шардирования защит в бизнес-логику, склейку шардов делает сервис. Типичный случай: Postgres и Mongo, сшитые в коде.
- Распределённое файловое или объектное хранилище. Нет модели данных и декларативного языка запросов — только «положи файл, возьми файл». Поиск по сути перебором. Пример с занятия: подсчёт объёма тайлового хранилища на 10 млн объектов занял полтора часа, и это только по метаданным.
- Реплицированный standby. Поточковая репликация даёт второй экземпляр для отказоустойчивости и чтения, но запросы не распределяются. Мощный ЦОД в Москве с тремя копиями — это одна база с репликами, а не РБД.

11. Зачем это нужно и чем платим (5.1)

Преимущества:

- Локализация: данные, которые порождаются и используются на площадке, там же и лежат — доступ быстрый.
- Доступность: локальный запрос переживает отказ глобального контура, брокер уводит сигнал на живой узел.
- Масштабируемость: новый узел добавляется без перенастройки остальных.
- Следование оргструктуре: чем выше уровень управления, тем меньше нужна детализация. Гендиректору не нужны счета на канцелярию по каждому филиалу, директору филиала — нужны.
- Разные режимы связанности на разных уровнях: жёсткие гарантии внутри контура, более слабые на аналитическом уровне.
- Снижение трафика при настоящем разделении данных: гоняются только служебные и синхронизационные данные, а не всё целиком.

Цена:

- Сложность фрагментации: правила работы с ключами приходится задавать на уровне системы для каждого фрагмента.
- Дорогая ошибка ключа. Локальный конфликт лечится, глобальное совпадение ключей — перебор данных, смена ключей и связей. Источники: потеря передачи или задержка регистрации ключа. Такие ситуации принимают как штатные и закладывают под них ресурс.
- Рост накладных расходов и трафика: данные могут гулять по сети до полной синхронизации даже там, где они не нужны.
- Больше цепочек передачи, а значит больше рисков канала связи.

Когда РБД оправдана: когда стоимость ошибки или задержки выше стоимости самой системы. Филиальная структура, работа по стране или миру, сценарии вроде продажи билетов, где ожидание ответа Москвы означает потерю сделки.

Отдельно рекомендовано прочитать 12 правил Криса Дейта для распределённых баз.

12. Классификация архитектур (5.1, 5.2)

Классификация многомерная: одна и та же система по разным осям попадает в разные классы. Четыре оси.

Ось 1. Что разделяют процессы.

- Shared memory — симметричные процессы с единым адресным пространством и общим буферным пулом. Параллелизм есть, распределения по сети нет. Годится, когда на одном узле под общей схемой держат несколько процессов под разные типы данных (документы + вектор, документы + граф).
- Shared disk — несколько экземпляров работают с одним комплектом файлов по сети хранения. Запись упирается в носитель, зато чтение распараллеливается. Нужен протокол когерентного кэша, иначе страницы будут постоянно вычитываться заново. Узкое место — внутренняя сеть. Пример: терабайты научных данных (замеры льдов), которые пишутся один раз и раздаются наружу по медленным внешним каналам.
- Shared nothing — у каждого узла своя память, диск и экземпляр, обмен только служебными сообщениями. Доминирующая модель, на ней работает большинство шардированных кластеров. Добавление ёмкости почти линейное: «почти» — потому что служебная информация растёт не только на новом узле, но и на всех остальных. Отсюда открытый вопрос: наращивать число нод или объём одной, учитывая что диски большей ёмкости дорожают нелинейно, а черепичная запись (SMR) сомнительна по надёжности при частой перезаписи.

Ось 2. Однородность схемы. Однородные системы — один движок, одна мажорная версия, одна схема: проще оптимизация, репликация, планировщик. Неоднородные допускают разные движки и локальные схемы, стыкуемые дополнительным слоем. Возникают из наследия (АСУ с зашитой базой, корпоративный учёт с историей за 20 лет, слияние компаний). Для профиля КИИ неоднородность — норма, иногда её закладывают в проект сразу.

Ось 3. Роли узлов.

- Клиент-сервер: один узел порождает запрос, другой исполняет и хранит. Чтобы это стало РБД, нужны несколько узлов и координатор либо балансировщик.
- Одноранговые (P2P): фиксированных ролей нет, каждый узел хранит фрагменты, принимает запросы и пересылает соседям то, на что не может ответить сам. Точкой входа может быть любой узел; на клиенте держат список узлов по удалённости, недоступный сменяется следующим. Если данные распределены, принявший запрос узел рассылает части соседям и собирает общий ответ.

Проблема P2P и мультимастера: расхождение версий и дублирование ключей, а решать конфликт некому. Разрешают по времени (кто раньше/позже), векторами версий или иным заданным правилом. Чистый одноранговый вариант без мастера опасен — два узла могут принять противоположные решения по одному объекту. Практический компромисс: одноранговая схема с автоназначением мастера, когда в каждый момент кто-то отвечает за окончательное решение, а при его отказе мастером становится другой узел.

Ось 4. Прозрачность схемы — см. следующий раздел.

13. Прозрачность (5.2)

Прозрачность — способность скрыть от приложения то, что база распределена. Для клиента не должно быть разницы. Пять свойств:

- Location transparency — скрыт узел хранения.
- Fragmentation transparency — скрыто то, что разные части данных лежат на разных узлах.
- Replication transparency — скрыто наличие копий.
- Concurrency transparency — скрыты чужие транзакции, всё выглядит как обычный ACID, без ручных распределённых блокировок.
- Failure transparency — отказ отдельного узла отдаётся как обычная локальная ошибка, без указания сбойного узла. Если из трёх узлов транзакция прошла на двух, третий догонит при восстановлении.

Принципиальный тезис занятия: если распределение разруливается на бэкенде — это не ошибка реализации, а ошибка логики. База данных склеивается с бизнес-логикой, слои перестают быть независимыми, дальше легаси и костыли. Полная прозрачность не бесплатна и достигается не всегда, но нарушать её «ради скорости» — типичный путь к неуправляемой системе, особенно когда изменения не документируются.

Здесь же теорема CAP: распределённая система не может одновременно давать линейную согласованность, доступность и устойчивость к разделению сети. Чем-то придётся жертвовать; алгоритмы-решения смотреть отдельно.

Для КИИ это не только про технику: временное решение, о котором забыли, стоит объекту сертификации, а худший исход нарушений — не потеря данных и денег, а человеческие жертвы и экологические последствия.

14. Фрагментация (9.1)

Около 90% трафика локализуется внутри подразделения, общим остаётся меньшая часть. Фрагмент удобен ещё и как самостоятельная единица резервирования: пять фрагментов по терабайту копируются параллельно и быстрее, чем одна база на пять терабайт. Иллюстрация масштаба: заливка 3 ТБ в университетское облако по сети оценивалась в ~147 дней, физическим диском в ноду — четверо суток.

Горизонтальная фрагментация — деление по кортежам (строкам). Объединение всех фрагментов должно покрывать исходное множество. Границу деления определяют эмпирически по структуре логов запросов: так, чтобы большая часть обращений попадала в один фрагмент. Обычно совпадает с административными и географическими границами.

Производная фрагментация — дочерние отношения режутся по тому же плану, что и родительские. Локальный администратор не имеет права нарезать базу дополнительно без согласования с глобальным уровнем: иначе централизованное обновление может не пережить самодеятельность, а с точки зрения ИБ появляется лишний вектор компрометации.

Вертикальная фрагментация — проекция отношения на подмножество атрибутов, данные о сущности лежат «слоями». Каждый фрагмент содержит ключ для сборки. Это не нарушение третьей нормальной формы: соблюдается условие соединения без потерь,

реконструкция идёт через natural join по ключу.

Отступление про natural join: он соединяет по совпадающим именам полей, а не по объявленным ключам. Поэтому требует строго индивидуализированных имён в каждой таблице, что само по себе ограничивает универсальность кода — на практике преподаватель использовал его считаное число раз.

Вертикальная фрагментация хорошо ложится на мандатный доступ: грантовая система выдаёт всё или ничего, мандатная позволяет выдать доступ к отдельным слоям данных.

Гибридная фрагментация — композиция горизонтальной и вертикальной в любом порядке. Если первичной была вертикальная, каждый слой может дальше делиться горизонтально независимо от других: структуру данных это не ломает.

Два вопроса, которые в КИИ выводят на выбор способа: «чей это объект» и «кому этот объект доступен».

Корректность фрагментации — три условия:

- Полнота — условия разделения покрывают всё исходное множество.
- Восстановимость — существует реляционное выражение, собирающее исходное отношение (объединение для горизонтальной, соединение для вертикальной). Именно это отличает РБД от набора параллельных баз, склеиваемых прикладным слоем.
- Непересечение — для горизонтальной кортеж попадает ровно в один фрагмент; для вертикальной атрибуты не дублируются, кроме ключа сборки.

15. Размещение фрагментов (9.1, 9.2)

Фрагментация отвечает на вопрос «как режем», размещение (allocation) — «где и в скольких экземплярах лежит».

- Неизбыточное размещение: каждый узел хранит один фрагмент. Плюсы — меньший объём хранения и отсутствие проблем согласования. Минус — при отказе узла фрагмент недоступен, и всё встало.
- Избыточное размещение: фрагменты на нескольких узлах. Плюс — доступность и надёжность, минус — нужен протокол согласования копий (кворум на запись: транзакция проходит при подтверждении большинства узлов либо всех, если система параноидальная).

Кроме географии и надёжности на разбиение влияют организационные и правовые требования: не хранить персональные данные в аналитическом контуре, не выводить технологический поток без агрегации, иметь локальную копию, достаточную для работы при обрыве связи с центром.

Важное практическое правило: модель данных с разграничением доступа согласуется с юридической службой до реализации. Есть подпись — работаем; нет — «технически всё верно, но закон нарушен».

Агрегаты выгодно держать избыточно: посчитали и отдали в центр — дальше за ними не ходят, лишнего трафика нет, персональных данных не уходит. Если центр остаётся без агрегатов, он начинает тянуть сырые данные, и нагрузка растёт экспоненциально. Эти же агрегаты обычно становятся экспортными данными для федеративного объединения разнородных баз.

16. Обработка распределённого запроса (9.1)

Клиент отправляет обычный SQL-запрос «чёрному ящику». Дальше:

- Декомпозиция — запрос переводится в дерево реляционной алгебры над глобальными отношениями.
- Подстановка определений фрагментов — дерево становится деревом над частями данных, каждая ветвь разрешается отдельно.
- Локализация — из обработки выбрасываются фрагменты, чьи предикаты противоречат предикату запроса (фрагмент чужого региона не опрашивается).
- Проталкивание (predicate pushdown) — селекции и обрезка столбцов переносятся как можно ближе к узлу с фрагментом, чтобы по сети шли уже отфильтрованные данные.
- Выбор места сборки — оптимизатор решает, где выполнять итоговое соединение: на одном из задействованных узлов, на наименее загруженном или в центре. Критерии: процессорные единицы, страницы памяти, объём передачи по сети (локальная сеть и глобальная считаются по-разному), а также требования надёжности и безопасности — перегон данных в чужой контур ради разгрузки должен быть запрещён.

17. Репликация (9.2)

- Репликация — поддержание копий одних и тех же данных на нескольких узлах, где изменение на одном рано или поздно отражается на других. Реплика — это копия фрагмента.

Три цели: доступность (при отказе главного одна из копий становится главной), сокращение издержек (запрос выполняется на ближайшей реплике), разделение нагрузки (тяжёлая аналитика уходит на выделенную реплику и не задерживает остальных).

Синхронная и асинхронная. При асинхронной транзакция считается зафиксированной на ведущем до подтверждения ведомого: быстро, но точка восстановления ненулевая — при отказе часть зафиксированных транзакций может не оказаться на реплике. Для критичных контуров (журнал переключения оборудования, ККМ) это недопустимо. При синхронной фиксация ждёт подтверждения от одной или нескольких реплик: commit не вернётся, пока журнал не достиг оговорённого рубежа. Выбор режима определяется важностью данных. Важно: на реплику передаётся журнал, а не сам запрос.

Физическая и логическая. Физическая копирует журнал предзаписи на уровне страниц, ведомый получается побайтовой копией; передача либо файлами журнала (потери в пределах размера сегмента), либо потоковая (канал занят постоянно, потери минимальны). Логическая декодирует изменения и применяет их как отдельные операции — ведомый может отличаться версией и в допустимых пределах схемой. Для аварийного переключения берут физическую, для интеграционных решений и витрин с урезанным набором данных — логическую.

Топологии:

- Ведущий — горячий резерв: пишем только в ведущий, ведомые получают журнал и могут обслуживать чтение. Конфликтов записи нет, переключение практически ручное. Базовый режим для оперативного контура.
- Мультимастер: запись на нескольких узлах. Производительность выше, но появляются конфликты — разрешаются по времени, по приоритету узла или вручную администратором; возможны расхождения в учёте.
- Каскадная: промежуточный узел-ретранслятор получает журнал от ведущего и раздаёт нижестоящим. Разгружает ведущего и экономит межплощадочный трафик.

18. Распределённые транзакции (10.1)

Каждый узел хранит свою часть данных и свой журнал. Координатор ведёт общую логику и принимает итоговое решение — commit или rollback.

Двухфазная фиксация (2PC). Фаза подготовки: координатор опрашивает участников, каждый готовый записывает в устойчивое хранилище состояние prepared — ресурсы

удержаны, самостоятельно ни откатить, ни зафиксировать нельзя. Фаза решения: если все ответили положительно, координатор рассылает commit; если хоть один ответил отказом или не ответил в срок — общий откат.

Главная беда 2PC — блокировка при отказе координатора после того, как участники вошли в prepared: узел не знает состояния координатора и висит с занятыми ресурсами. Лечат вмешательством администратора или временем жизни незавершённой транзакции, оба варианта чреваты расхождениями. Форсированные записи журнала и круговые сообщения на географически разнесённой системе слишком дороги. Именно из-за этой проблемы и вырос механизм обмена журналами: главный контур планирует транзакции, ведомые применяют журнал, и при сбое данные довосстанавливаются до консистентного состояния без тащенья всей механики транзакций на каждый узел.

Трёхфазная фиксация (3PC). Добавлено промежуточное явное состояние перед фиксацией, чтобы участник, не получивший решения, мог завершить транзакцию по тайм-ауту, а не блокироваться навечно. Что именно он делает при тайм-ауте — настраивается: оборвать без фиксации либо (для параноидальных систем, где важно, чтобы операция была зафиксирована хотя бы где-то) выполнить локальный commit. Плата — обязательный, подробно прописанный план восстановления: после подъёма узел может обнаружить изменения у части соседей и должен знать, что с этим делать.

19. Контроль параллелизма в распределённой базе (10.1)

Речь о слое поверх обычного контроля одиночной СУБД: управление параллелизмом, согласование реплик, поддержание целостности.

- Распределённый двухфазный захват блокировок. Транзакция делится на фазу накопления блокировок и фазу освобождения; блокировки удерживаются до конца, чтобы обеспечить восстановимость. У каждого узла свой локальный менеджер блокировок, блокируются только ресурсы затронутых фрагментов. Глобальность означает: субтранзакция не отпускает блокировку на одном узле, пока работа идёт на другом — иначе связанность ломается. Захват автономен на каждом узле, освобождение управляется централизованно. Из-за долгих удержаний тяжёлую аналитику выносят на отдельные узлы.
- Метки времени. Конфликт разрешается по глобальному правилу — приоритет раннего или позднего. Описанных схем много: правило Томаса для устаревшей записи, wait-die и другие. Сложность — синхронизация времени: в распределённой среде нельзя брать штампы с независимых часов, потому что за секунду проходят сотни транзакций, а запрос времени доходит по-разному. Отсюда логические часы (счётчики Лампорта) и гибридные методы. В режиме мультимастера внешние часы иногда используются как критерий победителя при сбое.

20. Облачные базы данных (10.1, 10.2)

- Облачная база данных — база, развёрнутая и управляемая в облачной среде, а не на локальных серверах организации; доступ и операции идут через сеть, инфраструктуру поддерживает провайдер.

Идея выросла как сервис для небольших организаций: у большой базы стоимость аренды быстро перекрывает стоимость собственной инфраструктуры, особенно при росте потребляемых ресурсов. Плюсы — быстро развернуть, быстро масштабировать, не заниматься железом, питанием и сетью.

Модели:

- Управляемая БД как сервис: провайдер ставит, обновляет, бэкапит и масштабирует; пользователю остаётся работа с данными и доступами.

- IaaS: берётся виртуальная машина, на неё ставится СУБД, администрирование целиком на заказчике — патчи, настройки, доступы, резервирование, масштабирование, ровно как на физическом сервере.
- Гибрид: часть данных в своей инфраструктуре, часть в облаке. Причины разные: миграция, требования законодательства, публикация данных, федерация.

Главный минус — инфраструктура не своя, и нужно абсолютное доверие провайдеру в вопросах надёжности и конфиденциальности. Никакой договор не вернёт утёкшие данные.

Облако не обязательно означает публичный интернет. Университетский пример: небольшой кластер, куда добавляются машины, а ресурсы нарезаются виртуалками, к ним подключаются хранилища разных классов (NVMe для быстрого, жёсткие диски для холодного). Плюсы такой организации — централизованный аудит и изоляция на уровне ИБ, независимость от конкретной железки (при отказе узла нагрузка уходит на другие мощности), быстрое клонирование и снос виртуальных машин, отсутствие простаивающего железа. Цена — сама прослойка виртуализации съедает порядка 20–25% ресурсов, и это закладывается в расчёт наравне с резервированием.

Про проектирование объектов КИИ прозвучало два подхода: сверху вниз (что нужно → как организовать → какие ресурсы) и снизу вверх от места размещения (тундра, слабое электричество, спутниковая связь, дорогая доставка оборудования). На практике проходов несколько, схема балансируется между «что надо» и «где стоим». Бюджет — отдельная ловушка: план на 130 млн против фактических 20 млн.

СВОДНЫЙ КОНСПЕКТ

Часть III. Безопасность

21. Безопасность баз данных: базовые понятия (11.1)

Нарушение правил доступа ломает два неочевидных свойства — конфиденциальность (люди видят лишнее) и целостность (зависимые части перестают разрешаться, структура рвётся в одном месте). Недоступность видна сразу, эти два — нет.

- Субъект доступа — сущность, от имени которой выполняется операция. Не обязательно человек: прикладная роль, служба репликации, задание резервного копирования. Пример с занятия: бэкап выполняется от имени задания, а не администратора, потому что у администратора прав на сами данные может не быть вовсе. Обратный плохой случай — когда за одним субъектом стоит множество людей (внешнее приложение под общей учёткой): отследить автора действия почти невозможно. Архитектура соединений — тоже часть модели доступа: доступ из внутренней сети и из внешней должен разграничиваться.
- Объект доступа — то, на что направлена операция: кластер, база, схема, таблица, столбец, строка, последовательность, процедура, представление. Гранулярность — минимальная единица, на которую можно наложить правило.
- Право — разрешённый вид операции над объектом.
- Политика — правило, которое для пары «субъект — операция над объектом» выдаёт разрешение или запрет; может опираться на данные самой записи (пользователь правит только свои строки).

Где размещать политики: в правильно построенной распределённой базе — на верхнем слое, чтобы декомпозиция запроса сразу их учитывала и отсекала лишнее наверху. Выносить это на бэкенд значит «очень быстро потерять». Оговорка: разграничение субъектов имеет смысл только там, где субъекты действительно разные; если всё идёт от одной учётки, политики бессмысленны.

Три свойства хорошей политики доступа: полнота (для каждой операции есть явное разрешение или явный запрет по умолчанию), непротиворечивость (два правила не могут одновременно разрешать и запрещать без заданного порядка разрешения; что главнее — личный доступ или групповой, однозначного ответа нет, и это должно быть прописано), проверяемость (аудитор может ответить, кто и почему получил разрешение). Отсюда требование: журнал должен переживать отказ всего остального, иначе в нём нет смысла. В распределённой системе добавляется риск, что на разных узлах одной роли прописаны разные права.

Три глагола, которые путают: идентификация — заявление субъекта о том, кто он (имя, идентификатор, сертификат, ключ); аутентификация — проверка этого заявления (пароль, подпись, сертификат, токен); авторизация — решение, какие операции данному субъекту разрешены на данном объекте. Аутентификация может быть общей, а авторизация в настоящей РБД — своя на каждом узле, и это нормально: на одной базе лежат пациенты, на другой персональные данные студентов, и права там разные.

22. Модели управления доступом (11.1, 11.2)

Дискреционная (DAC). Решение принадлежит владельцу объекта. В основе матрица «субъект — объект — права», управляется GRANT и REVOKE. Создатель объекта становится владельцем и может выдавать права, в том числе отозвать их у себя. В Postgres право удалить или изменить объект не является выдаваемой привилегией — оно остаётся у

владельца, передаётся только сменой владельца. Суперпользователь обходит все ограничения, поэтому раздавать администраторам суперправа — плохая практика; администратор базы не должен управлять ролями пользователей.

GRANT имеет два смысла: выдать привилегию на объект и выдать членство в роли. Группа — это по сути роль без логина. Отдельная ловушка — WITH GRANT OPTION: субъект может передавать доступ дальше, и при отзыве у него выданные им права могут остаться. Отзыв обязан учитывать зависимые выдачи. Правильная схема для временной рабочей группы: создать роль, дать ей права, разрешить руководителю добавлять в неё сотрудников, а по завершении отозвать роль целиком — доступ пропадёт у всех разом.

Представления как ограничение видимости. Субъекту выдаётся грант на представление вместо базовой таблицы, прямое обращение к таблице запрещается; лишние столбцы просто не попадают в выборку. WITH CHECK OPTION ограничивает вставку и изменение: операция проходит, только если предикат истинен, — иначе субъект мог бы вставить запись, которую сам не увидит, но которая отобразится у другого. Важно: представление не криптография. Суперпользователь его обходит, поэтому разграничение не отменяет журналирование.

Row-level security. Горизонтальное ограничение на уровне строк без заведения отдельного представления под каждую роль (при добавлении роли пришлось бы плодить новый комплект). Работает поверх обычного доступа к таблице; если политик нет — запрет по умолчанию. Исключения: владелец таблицы, суперпользователь и роли с BYPASSRLS. Политика задаётся выражением: USING для чтения, WITH CHECK для вставки и изменения (если не задан, совпадает с USING). Разрешающие политики складываются по «или», ограничивающие — по «и». Остаточный риск: по ошибке внешнего ключа можно сделать вывод о существовании скрытых строк.

Мандатная модель (Белла — Лападулы), конфиденциальность. Классы доступа жёстко разделены: субъект не читает объекты класса выше своего («не читать вверх») и не пишет в документы класса ниже своего («не писать вниз»), чтобы секретное не утекло в несекретное. Двойственная ситуация: если поле документа имеет класс выше класса субъекта, он его не видит, но формально может туда писать — система не даст затереть данные, а создаст низкоуровневую версию значения поля, при этом обладатель высокого доступа видит полную картину. Модель сложна в реализации; Postgres мандатное разделение не поддерживает, и наложить его поверх существующей грантовой схемы означает разводить контуры. Если задача возникла всерьёз — лучше один раз перейти на систему, изначально ориентированную на мандатный доступ.

Модель Биба, целостность. Зеркальная предыдущей: «читать вниз нельзя, писать вверх нельзя». Высокоцелостный субъект не читает данные низкой целостности, низкоцелостный не изменяет высокоцелостные. Отчёт регулятору не берёт цифры из непроверенного документа; нельзя глушить реактор по срабатыванию одного непроверенного датчика. Для КИИ целостность зачастую важнее конфиденциальности. На практике в одном центре обе модели уживаются: Биба — для контура управления, Белла — Лападулы — для контура расследования инцидентов.

Ролевая модель (RBAC). Роль — устойчивый набор полномочий, соответствующий должностной функции. Права назначаются роли, роль — пользователю; смена должности сводится к смене роли, а не к переписыванию сотен грантов. Иерархия ролей позволяет наследовать права (старший дежурный наследует дежурного и добавляет контрольные функции). Отсюда же разделение обязанностей: создаёт задачу и проверяет её разные роли. У одного человека может быть несколько ролей, но противоречить друг другу они не должны — приоритета у ролей обычно нет, и конфликт разрешится непредсказуемо. В наследуемых иерархиях унаследованное разрешение можно заменить (например, «создавать наряд» на «контролировать наряд»).

Организационное

- Индивидуальный проект: под капотом должен скрывать функционал первого, второго и третьего заданий — структура данных по своей теме исследования. Обязательно обосновать, что и почему разделяется, как это связано на уровне базы и в чём именно распределённость. Plusом идёт собственная надстройка, склеивающая две разные СУБД в единую точку обращения; ещё больший плюс — своя упрощённая база в составе распределённого контура.
- Прочитать самостоятельно: 12 правил Криса Дейта, теорема CAP и алгоритмы на её основе, теорема о горизонтальной фрагментации кортежей, правило Томаса и схемы wait-die.
- Защита проекта — с рассказом и показом работы; заявку на тему подать заранее.